STUDENT TASK MANAGER

A DevOps Laboratory Project Report

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

MOHAMMED SOHAIL (24261A05A0)

PERUMANDLA TEJASHWIN (24261A05A8)

VINJAMUR KAUSHIK (24261A05C6)

Under the guidance of

K. SUNITHA

Faculty In-charge

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)

Gandipet, Hyderabad-500075, Telangana, India (Autonomous)

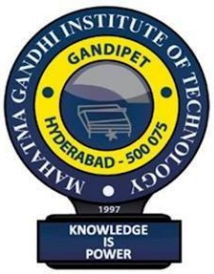
2026–2027

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)

Gandipet, Hyderabad-500075, Telangana, India (Autonomous)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the DevOps laboratory project report entitled **"STUDENT TASK MANAGER"** is being submitted by **MOHAMMED SOHAIL** bearing Roll No. **24261A05A0**, **PERUMANDLA TEJASHWIN** bearing Roll No. **24261A05A8** and **VINJAMUR KAUSHIK** bearing Roll No. **24261A05C6** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering**, and is a record of bonafide work carried out by them in the Department of Computer Science and Engineering, Mahatma Gandhi Institute of Technology, during the academic year 2026–2027, under our guidance and supervision.

Faculty In-charge
K. SUNITHA
Department of CSE, MGIT

Head of the Department
RAJNIKANTH
Department of CSE, MGIT

DECLARATION

We hereby declare that the work reported in this project report entitled **"STUDENT TASK MANAGER"** is a record of work carried out by us in the Department of Computer Science and Engineering, Mahatma Gandhi Institute of Technology, Hyderabad, as part of the DevOps laboratory during the academic year 2026–2027. The application, configuration files, tests and evidence described in this report were produced by our team. Wherever material from documentation or other sources has been used, it has been duly acknowledged and referenced. This work has not been submitted elsewhere for the award of any degree or diploma.

MOHAMMED SOHAIL (24261A05A0)

PERUMANDLA TEJASHWIN (24261A05A8)

VINJAMUR KAUSHIK (24261A05C6)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without mentioning the people who made it possible. We take this opportunity to thank all those whose guidance and encouragement helped us complete this project.

We express our sincere thanks to the **Principal, Mahatma Gandhi Institute of Technology**, for providing the facilities and environment needed to carry out this work.

We are grateful to **Rajnikanth**, Head of the Department of Computer Science and Engineering, MGIT, for the constant support and encouragement extended to us throughout the DevOps laboratory.

We are deeply thankful to our faculty in-charge, **K. Sunitha**, Department of Computer Science and Engineering, MGIT, for the guidance, valuable suggestions and continuous support given to us, which helped us understand and apply DevOps practices in this project.

Finally, we thank all the faculty and staff of the Department of Computer Science and Engineering, and our families and friends, who helped us directly or indirectly in completing this project.

MOHAMMED SOHAIL (24261A05A0)

PERUMANDLA TEJASHWIN (24261A05A8)

VINJAMUR KAUSHIK (24261A05C6)

ABSTRACT

DevOps unifies software development and operations through collaboration, automation and continuous feedback. This project applies the core DevOps tool chain to **Student Task Manager**, a framework-free web application built with HTML, CSS and vanilla JavaScript that helps students organise their tasks. The application supports adding, completing and deleting categorised tasks, live task statistics, a persistent dark/light theme and a responsive layout. To create realistic user flows for automated testing, it was extended with a client-side authentication layer providing registration, login, logout, protected pages, profile editing, password change and separate task lists for each user. All data is stored in the browser's localStorage; there is no backend server or database, and the authentication is explicitly a demonstration-level mechanism rather than production-grade security.

Source code was managed with **Git and GitHub** using a feature-branch workflow, with three features integrated through pull requests and the second-phase work combined on an integration branch. A **Jenkins** declarative pipeline on a Windows agent checks out the repository, validates that the required files exist using batch commands, and archives the files with fingerprints; build #14 completed all stages successfully. The application was containerised with **Docker** using the nginx:alpine image, published on host port 8080, and orchestrated with **Docker Compose** together with a Selenium standalone Chrome service and a Python test-runner service, using profiles, service-name networking and a health check. An automated **Selenium WebDriver and Pytest** suite of 16 end-to-end test cases, designed from a shared contract before the implementation was written, covers signup validation, login, access control, profile management, task persistence, per-user task isolation and theme persistence. In the observed run, **all 16 tests passed in 34.01 seconds**.

The report documents the planning, implementation, CI pipeline, containerisation, testing, deployment and operation of the project with evidence from the repository, Jenkins, Docker and the test runs, and identifies its limitations and future enhancements such as backend authentication, a database, cloud deployment and running the browser tests within the CI pipeline.

Keywords: *DevOps, Git, GitHub, Jenkins, continuous integration, Docker, Docker Compose, Nginx, Selenium, Pytest, localStorage, test automation*

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 1
1.1 Introduction to DevOps 1
1.2 Motivation for the Project 2
1.3 Problem Statement 2
1.4 Project Objectives 2
1.5 Scope of the Project 3
1.6 Overview of Student Task Manager 3
1.7 Technologies Used 4
CHAPTER 2: PROJECT PLANNING AND VERSION CONTROL 6
2.1 Project Planning 6
2.2 Team Organisation and Work Distribution 6
2.3 Git and GitHub 7
2.4 Repository Structure 8
2.5 Branching Strategy 9
2.6 Commit History 10
2.7 Pull Requests and Code Integration 12
2.8 Merge Workflow in the Second Phase 13
CHAPTER 3: APPLICATION DEVELOPMENT 14
3.1 Application Architecture 14
3.2 Frontend Implementation 15
3.3 Task Management Operations 16
3.4 localStorage Persistence 17
3.5 Authentication Implementation 18
3.6 Profile Management and Password Change 21
3.7 Theme Switching 22
3.8 User-Specific Task Storage 24
CHAPTER 4: CONTINUOUS INTEGRATION USING JENKINS 25
4.1 Introduction to Continuous Integration 25
4.2 Jenkins Installation and Configuration 25
4.3 Jenkins Job Configuration 26
4.4 The Jenkinsfile and Declarative Pipeline 26
4.5 Build Execution 28
4.6 Console Output 29
4.7 Build Success and Failure Handling 29
4.8 Extended Jenkinsfile on the Integration Branch 30
4.9 Jenkins CI and the Selenium Test Suite 32

CHAPTER 5: CONTAINERISATION USING DOCKER 33
5.1 Introduction to Containerisation 33
5.2 Why Docker Was Used 33
5.3 Dockerfile Explanation 33
5.4 Port Mapping 34
5.5 Docker Image Creation and Container Execution 34
5.6 Docker Compose 35
5.7 Running Containers 37
5.8 Test-Runner Image 38
CHAPTER 6: AUTOMATED TESTING USING SELENIUM AND PYTEST 39
6.1 Introduction to Software Testing 39
6.2 Selenium WebDriver 39
6.3 Pytest 39
6.4 Dockerised Testing Environment 40
6.5 Test Execution Workflow 41
6.6 Test Case Design 41
6.7 Test Results 42
6.8 Analysis of Results 45
CHAPTER 7: DEPLOYMENT AND RELEASE 46
7.1 Deployment Approaches Used 46
7.2 Application Deployment Using Docker 46
7.3 Nginx Hosting 47
7.4 Port Mapping and Access Through localhost 47
7.5 GitHub Pages Deployment 47
7.6 Release Workflow 48
7.7 Build Artifacts 48
CHAPTER 8: OPERATIONS AND MONITORING 49
8.1 Running Containers 49
8.2 Docker Compose Lifecycle 49
8.3 Container Health and Application Availability 49
8.4 Jenkins Build History and Status Monitoring 50
8.5 Logs and Troubleshooting 50
8.6 Common Problems Encountered and Solutions 50
8.7 Limitations of the Current Monitoring Setup 51
CHAPTER 9: CONCLUSION AND FUTURE SCOPE 53
9.1 Project Achievements 53
9.2 DevOps Concepts Demonstrated 53
9.3 Technical Learning Outcomes 54
9.4 Limitations 54

9.5 Future Enhancements 54
9.6 Conclusion 55
REFERENCES 56

LIST OF FIGURES

Figure 1.1: Overview of the DevOps workflow implemented in the project 4
Figure 2.1: GitHub repository overview showing the main branch, commit count and github-pages deployments 8
Figure 2.2: GitHub branch overview showing the default and active branches 10
Figure 2.3: Branch and merge history reconstructed from the Git log 10
Figure 2.4: Merged pull request #3 (add tasks categories) on GitHub 12
Figure 3.1: Client-side architecture of Student Task Manager 14
Figure 3.2: Task dashboard after login, with personalised greeting and empty task list 17
Figure 3.3: Adding a new task with a category 17
Figure 3.4: Signup page (screenshot not available in the evidence folder) 20
Figure 3.5: Login page in dark theme (email address redacted) 20
Figure 3.6: Profile page with name, read-only email (redacted) and change-password form 22
Figure 3.7: Task dashboard in light theme with two pending tasks 23
Figure 3.8: Task dashboard in dark theme with the same tasks 23
Figure 4.1: Jenkins dashboard showing the Student-Task-Manager-CI job and its last success and failure 26
Figure 4.2: Pipeline stages executed in build #14 and their durations 27
Figure 4.3: Stage view of successful build #14 29
Figure 4.4: Console output of build #14 ending with Finished: SUCCESS 29
Figure 4.5: Extended Jenkinsfile on the integration branch (Validate stage excerpt) 31
Figure 5.1: docker-compose.yml with the web, selenium and tests services 35
Figure 5.2: Docker Compose services, network and published ports 37
Figure 5.3: docker ps showing the healthy Selenium container and the web container with 8080->80 37
Figure 5.4: Output of docker compose up --build (screenshot not available) 38
Figure 6.1: Pytest session header and per-test results (16 items collected) 42
Figure 6.2: Short test summary showing 16 passed in 34.01 seconds 43
Figure 7.1: Application opened at http://localhost:8080 with the address bar visible (screenshot not available) 47

LIST OF TABLES

Table 1.1: Technologies used in the project 4
Table 2.1: Team members, responsibilities and supporting evidence 6
Table 2.2: Branches in the remote repository 9
Table 2.3: Complete commit history (git log --all) 11
Table 2.4: Pull requests merged into main 12
Table 3.1: Application source files and their roles 15
Table 3.2: localStorage keys used by the application 18
Table 3.3: Validation rules and messages (from the contract, verified in auth.js) 19
Table 4.1: Summary of pipeline stages 28
Table 4.2: Executed Jenkinsfile (main) compared with the extended Jenkinsfile 31
Table 5.1: Docker and Docker Compose commands used 34
Table 5.2: Docker Compose services (verified from docker-compose.yml) 36
Table 6.1: Test environment 40
Table 6.2: Selenium test cases and results 43
Table 6.3: Test IDs and Pytest node identifiers 44
Table 6.4: Summary of results by module 44
Table 7.1: GitHub Pages deployment compared with local Docker deployment 46
Table 8.1: Docker Compose lifecycle 49
Table 8.2: Problems encountered and solutions implemented 51
Table 9.1: DevOps concepts and how they were demonstrated 53

CHAPTER 1: INTRODUCTION

1.1 Introduction to DevOps

DevOps is a set of practices, cultural values and tools that brings software development (Dev) and IT operations (Ops) together so that software can be built, tested, released and operated quickly and reliably. In a traditional organisation, developers write code and then hand it to a separate operations team that deploys and maintains it. This hand-over is often slow and error-prone, because the two groups use different environments, tools and priorities. DevOps replaces the hand-over with a shared, automated workflow in which every change moves through the same repeatable steps.

The DevOps life cycle is commonly described as a continuous loop of **plan, code, build, test, release, deploy, operate and monitor**. Each stage is supported by tools: version control systems such as Git record every change; hosting platforms such as GitHub support collaboration through branches and pull requests; continuous integration (CI) servers such as Jenkins automatically check each change; container platforms such as Docker package an application together with its runtime so that it behaves the same on every machine; and test automation frameworks such as Selenium verify that the application still works from the user's point of view.

The key principles that guide DevOps practice are:

- **Automation** – repetitive tasks such as checking files, building images and running tests are scripted so that they are fast and consistent.
- **Collaboration** – team members work on separate branches and integrate their work through reviewed pull requests instead of editing one shared copy.
- **Continuous integration** – every change is integrated into a shared repository frequently and verified automatically.
- **Reproducible environments** – infrastructure and runtime configuration are described as code (for example a `Dockerfile` or a `docker-compose.yml`), so any team member can recreate the same environment.
- **Fast feedback** – automated checks and tests tell developers quickly whether a change has broken something.

1.2 Motivation for the Project

DevOps concepts are best understood by applying them to a real codebase. However, a large application would shift the focus of a laboratory project towards application features rather than the DevOps workflow. The team therefore chose a deliberately small, framework-free web application, **Student Task Manager**, as the vehicle for demonstrating the DevOps tool chain. A task manager is familiar to every student, its behaviour is easy to verify visually, and it can be extended in small, clearly separated increments – which makes it ideal for practising feature branches, pull requests and automated testing.

As the project progressed, it became clear that the original application (adding, completing and deleting tasks) offered too few user flows to test meaningfully with Selenium. An authentication layer – signup, login, logout and profile management – was therefore added. These flows contain form validation, page redirects and per-user data, which provide realistic scenarios for browser automation while keeping the application entirely client-side.

1.3 Problem Statement

Students frequently juggle assignments, laboratory records, examinations and personal commitments, and benefit from a simple tool to track them. From the DevOps perspective, the problem addressed by this project is broader: **how can a small team develop such an application collaboratively, verify it automatically, package it so that it runs identically on any machine, and test its behaviour through a real browser, using industry-standard DevOps tools?** Without a disciplined workflow, team members overwrite each other's changes, configuration differs between machines, and regressions go unnoticed until a demonstration.

1.4 Project Objectives

The objectives of the project are:

1. To develop a responsive, framework-free Student Task Manager web application using HTML, CSS and JavaScript, with tasks persisted in browser localStorage.
2. To extend the application with client-side authentication (registration, login, logout, protected pages, profile editing and password change) and per-user task lists.
3. To manage source code with Git and GitHub using feature branches and pull requests rather than committing directly to the main branch.

4. To configure a Jenkins declarative pipeline that checks out the repository, validates that required files exist and archives build artifacts with fingerprints.
5. To containerise the application with Docker using an Nginx image, and to orchestrate the application and its test environment with Docker Compose.
6. To design and execute an automated Selenium WebDriver test suite with Pytest, running against the containerised application in a Dockerised Chrome browser.
7. To document the complete workflow, evidence and results in a formal project report.

1.5 Scope of the Project

The scope of the project is defined as follows:

- **In scope:** a static, client-side web application; browser localStorage for accounts, sessions, tasks and theme preference; Git/GitHub version control with branches and pull requests; a Jenkins CI pipeline on a Windows agent; Docker and Docker Compose for local deployment and testing; a 16-case Selenium/Pytest suite; static hosting through GitHub Pages.
- **Out of scope:** a backend server, a database server, server-side authentication, production-grade password security, cloud deployment of containers, and advanced monitoring or observability tooling. These are discussed as future enhancements in Chapter 9.

It is important to state clearly that the authentication implemented in this project is a **demonstration-level** mechanism. Accounts and sessions are stored in the browser of the user, and passwords are transformed with a simple, non-cryptographic hash. It is suitable for a classroom demonstration of testable user flows, but it is not secure enough for real accounts or sensitive data.

1.6 Overview of Student Task Manager

Student Task Manager is a multi-page static web application consisting of four HTML pages (`signup.html`, `login.html`, `index.html` and `profile.html`), one shared stylesheet (`style.css`) and two JavaScript files (`auth.js` and `script.js`). A user first creates an account, then logs in to reach the task dashboard. On the dashboard the user can add a task with a category (College, Personal, Work or Other), mark tasks as completed, delete tasks, and view live statistics (total, completed and pending tasks). The user may switch between light and dark

themes, and the choice is remembered. A profile page allows the user to change the display name and password. Each user's tasks are stored under a separate key, so different accounts in the same browser do not see each other's tasks.

Figure 1.1 summarises the overall DevOps workflow that was built around the application. Each element of the figure is described in detail in the following chapters.

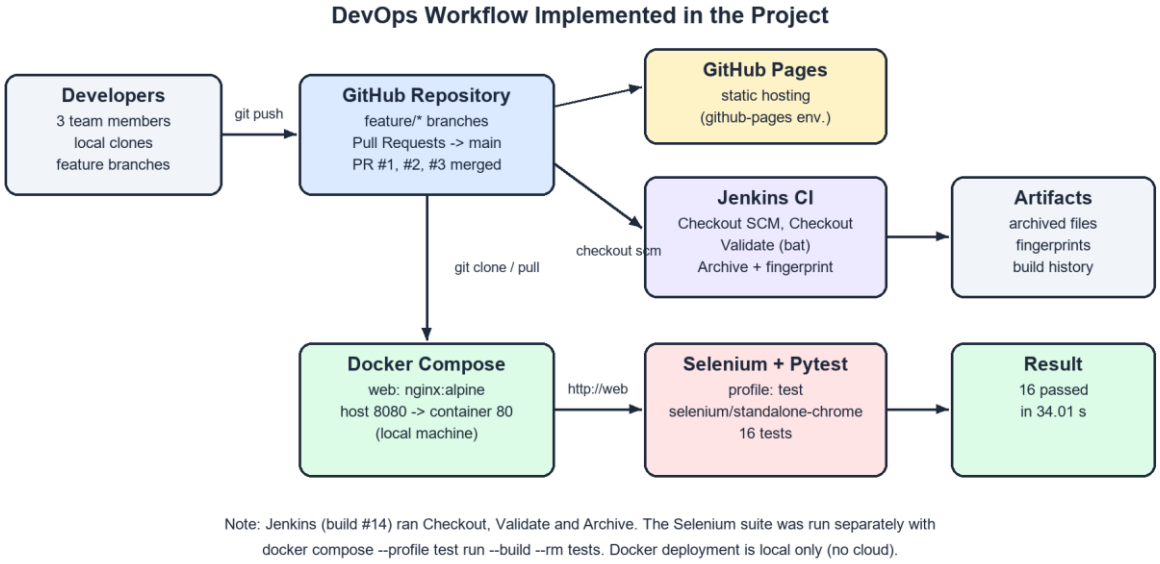


Figure 1.1: Overview of the DevOps workflow implemented in the project

1.7 Technologies Used

Table 1.1 lists the technologies used in the project and the purpose of each. All versions noted are those observed in the project files or in the captured evidence.

Table 1.1: Technologies used in the project

Technology	Category	Purpose in this project
HTML5	Frontend	Structure of the four pages: signup, login, task dashboard and profile
CSS3	Frontend	Styling, CSS custom properties for light/dark themes, responsive layout
JavaScript (ES6, vanilla)	Frontend	Task logic, client-side authentication, theme switching, DOM rendering
Browser localStorage	Client storage	Persists users, current session, per-user tasks and theme on the user's device
Git	Version control	Tracks changes; feature branches and merges
GitHub	Collaboration / hosting	Remote repository, pull requests, GitHub Pages static hosting

Technology	Category	Purpose in this project
Jenkins	Continuous integration	Declarative pipeline: Checkout, Validate, Archive on a Windows agent
Docker / nginx:alpine	Containerisation	Packages the static site into an Nginx web-server image
Docker Compose	Orchestration	Runs web, Selenium and test-runner services on one network
Selenium (standalone-chrome)	Test automation	Remote Chrome browser driven through WebDriver
Python 3.12, Pytest 8.4.2	Test framework	Test runner, fixtures and assertions (selenium>=4.20,<5.0)
Visual Studio Code	Development tool	Editing source files and configuration

CHAPTER 2: PROJECT PLANNING AND VERSION CONTROL

2.1 Project Planning

The project was planned in two phases. In the **first phase** (September 2026) the team built the base application and the initial DevOps workflow: the core task features, task statistics, task categories, a dark/light theme, GitHub Pages hosting and a Jenkins pipeline. In the **second phase** (late October to 1 November 2026) the team prepared for the laboratory evaluation by adding authentication, Docker and Docker Compose, an automated Selenium test suite, an extended Jenkinsfile and an updated README.

For the second phase the team wrote a shared planning document, `00 shared context.md`, which every member (and every AI coding assistant used by the members) was required to read first. It defines the current state of the repository, the division of work, and – most importantly – a **contract** that allowed two members to work in parallel. The contract fixes the page names, the HTML element IDs used as Selenium locators, the exact validation messages, the localStorage keys and the Docker URLs. Because the contract was agreed in advance, the Selenium tests could be written against it before the authentication code was finished.

The evaluation items listed in the shared context were: (1) Docker deployment demonstration, (2) Selenium testing demonstration, (3) an updated README file, and (4) a hard copy of the project report.

2.2 Team Organisation and Work Distribution

The work was divided into three responsibilities. Table 2.1 shows each member's responsibilities together with the repository evidence that supports the attribution. Commits are attributed through the Git author accounts listed in Section 2.6.

Table 2.1: Team members, responsibilities and supporting evidence

Member	Roll No.	Responsibilities	Repository evidence
Vinjamur Kaushik	24261A05C6	Project report and documentation; inspection and documentation of the DevOps workflow; evidence collection; initial project setup; Jenkins job configuration	Initial setup (e43f4a5); Jenkins pipeline fixes (853cd30, 55a6539, 559ce30); README updates; Jenkins build #14 "Started by Kaushik Vinjamur"; screenshots

Member	Roll No.	Responsibilities	Repository evidence
Perumandla Tejashwin	24261A05A8	Task statistics and categories; authentication implementation (signup, login, logout); profile management and password change; client-side auth integration	c81f73a (statistics, PR #1); 1452f30 (categories, PR #3, merged by the same account); e03b8f1 (auth, branch feature/auth)
Mohammed Sohail	24261A05A0	Dark-mode feature; Dockerfile and Docker Compose; Selenium test implementation; Jenkins pipeline extensions; README updates; branch integration	Six dark-mode commits (PR #2); f17a511 and 9ee8c97 (feature/docker-selenium); merge ef9e7a6; Docker Hub tag sohails2409/student-task-manager

2.3 Git and GitHub

Git is a distributed version control system. Every developer has a complete copy of the repository, including its history, on the local machine. Changes are recorded as commits; each commit has a unique SHA-1 identifier, an author, a timestamp and a message. Branches are lightweight, movable pointers to commits, which allow several lines of development to proceed independently and later be merged.

GitHub is a hosting platform for Git repositories. It adds a web interface, access control, pull requests with review and discussion, deployment environments such as GitHub Pages, and integration points for CI servers. The project repository is hosted publicly at `https://github.com/vinjamurkaushik-web/student-task-manager`. Figure 2.1 shows the repository overview at the time of capture: the default branch `main` with 18 commits, six branches, and a `github-pages` deployment environment with eight deployments.

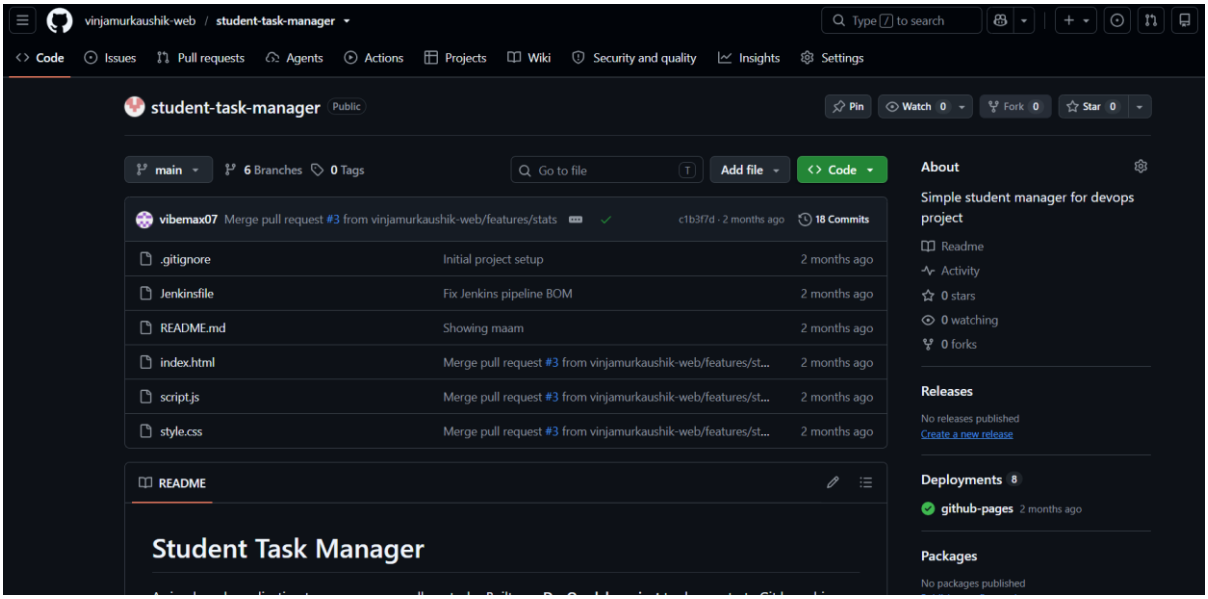


Figure 2.1: GitHub repository overview showing the main branch, commit count and github-pages deployments

The basic commands used throughout the project were:

Listing 2.1: Typical Git commands used in the feature-branch workflow

```
git clone https://github.com/vinjamurkaushik-web/student-task-manager.git
git checkout -b feature/auth          # create and switch to a feature branch
git add auth.js login.html signup.html profile.html
git commit -m "Add user authentication and profile management"
git push -u origin feature/auth       # publish the branch to GitHub
git log --all --oneline --graph       # inspect history across all branches
```

2.4 Repository Structure

The final repository on the integration branch `feature/auth-selenium-integration` contains the following files. The application files are at the repository root, so that both GitHub Pages and the Nginx container can serve them directly.

Listing 2.2: Repository structure on feature/auth-selenium-integration

```
student-task-manager/
|-- index.html            # Protected task dashboard
|-- login.html            # Login page
|-- signup.html           # Account creation page
|-- profile.html          # Profile and password page (protected)
|-- style.css             # Shared styles, light/dark themes, responsive rules
|-- script.js             # Task management and theme behaviour
|-- auth.js               # Shared client-side authentication logic
|-- Dockerfile            # nginx:alpine image for the static site
|-- .dockerignore         # Files excluded from the web image
|-- docker-compose.yml    # web, selenium and tests services
|-- Jenkinsfile           # Declarative pipeline (Windows agent, bat)
|-- README.md             # Project guide
|-- 00 shared context.md  # Team plan and contract
|-- docker.md             # Brief for the Docker/Selenium work
'-- tests/
    |-- Dockerfile        # python:3.12-slim test-runner image
    |-- requirements.txt  # pytest and selenium
    |-- conftest.py       # Remote browser fixture and helpers
    |-- test_signup.py    # TC01-TC06
    |-- test_login.py     # TC07-TC11
    |-- test_profile.py   # TC12-TC13
    '-- test_tasks.py     # TC14-TC16
```

2.5 Branching Strategy

The team followed a **feature-branch workflow**. The `main` branch represents the stable version of the application. Each new feature is developed on its own branch created from `main`; when the feature is complete it is integrated into `main` through a pull request. This keeps unfinished or experimental work away from the stable branch and makes every integration step visible and reviewable. The shared context explicitly states the rule: "One branch per person. Never commit straight to main."

Table 2.2 lists the branches present in the remote repository, and Figure 2.2 shows the GitHub branch overview.

Table 2.2: Branches in the remote repository

Branch	Purpose	Status (at time of capture)
main	Stable default branch	Contains PR #1, #2 and #3
features/stats	Task statistics, later task categories	Merged via PR #1 and PR #3
feature/dark-mode	Dark/light theme toggle	Merged via PR #2 (one later README commit remains on the branch)
feature/auth	Signup, login, logout, profile, per-user tasks	1 commit ahead of main; integrated into feature/auth-selenium-integration

Branch	Purpose	Status (at time of capture)
feature/docker-selenium	Docker, Compose, Selenium suite, Jenkinsfile, README	Integrated into feature/auth-selenium-integration
feature/auth-selenium-integration	Integration of auth with Docker/Selenium	5 commits ahead of main; no pull request to main yet

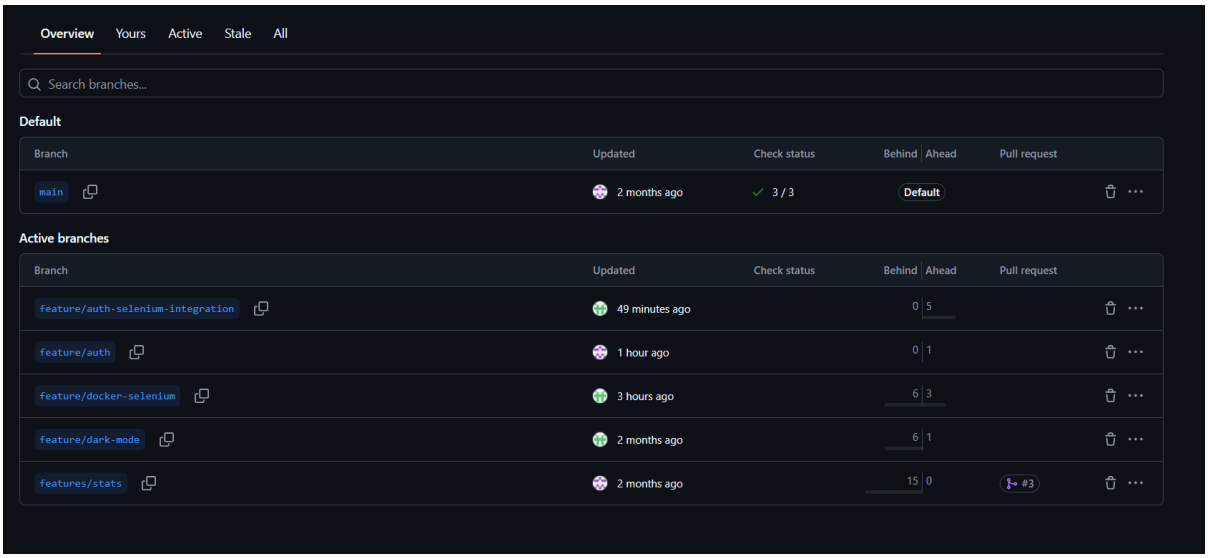


Figure 2.2: GitHub branch overview showing the default and active branches

Figure 2.3 is a simplified drawing of the actual branch and merge history, reconstructed from `git log --all --graph`. It shows how each feature branch left `main` and returned through a pull request, and how the two second-phase branches were combined on an integration branch.

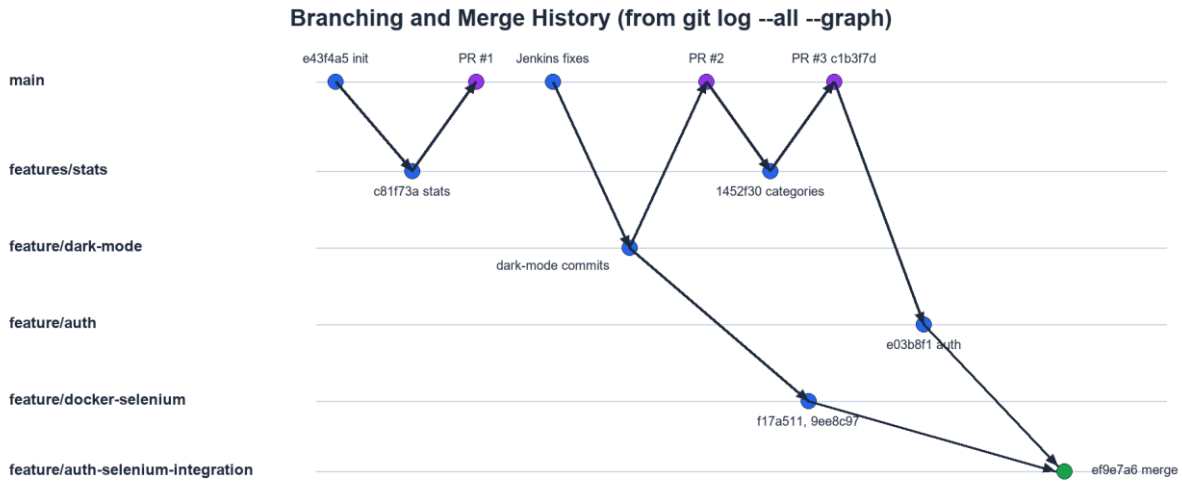


Figure 2.3: Branch and merge history reconstructed from the Git log

2.6 Commit History

Table 2.3 lists every commit in the repository across all branches, as reported by `git log --all`. The table is generated directly from the repository and has not been edited, apart from

omitting e-mail addresses. Author accounts map to team members as follows: **Kaushik192006** and **Kaushik2** – Vinjamur Kaushik; **vibemax07** and "Perumandla Tejashwin" – Perumandla Tejashwin; **RenNeedsSleep** – Mohammed Sohail.

Table 2.3: Complete commit history (git log --all)

Commit	Date	Author account	Message
ef9e7a6	2026-11-01 21:10	RenNeedsSleep	Merge auth and Selenium branches
e03b8f1	2026-11-01 20:35	vibemax07	Add user authentication and profile management
9ee8c97	2026-11-01 18:58	RenNeedsSleep	final changes before merge
f17a511	2026-11-01 18:47	RenNeedsSleep	selenium
c1b3f7d	2026-09-11 11:40	Perumandla Tejashwin	Merge pull request #3 from vinjamurkaushik-web/features/stats
738ef5d	2026-09-11 11:36	RenNeedsSleep	made some final changes
1452f30	2026-09-11 11:36	vibemax07	add tasks categories
c813f78	2026-09-11 11:10	Kaushik192006	Showing maam
c1c6701	2026-09-11 09:36	Kaushik192006	New change now
b9f1bd6	2026-09-10 20:05	Kaushik2	Merge pull request #2 from vinjamurkaushik-web/feature/dark-mode
949a030	2026-09-10 19:40	RenNeedsSleep	docs: update README to document dark mode feature and branch
6f874b7	2026-09-10 19:39	RenNeedsSleep	refactor: simplify theme label sync, remove redundant DOMContentLoaded listener
c058ddf	2026-09-10 19:38	RenNeedsSleep	fix: apply saved theme before paint to prevent FOUC
9c810e6	2026-09-10 19:37	RenNeedsSleep	add toggleTheme() function with localStorage persistence
b2f7d49	2026-09-10 19:34	RenNeedsSleep	add CSS variables for light and dark theme colours
155a748	2026-09-10 19:30	RenNeedsSleep	add dark mode toggle button to header
2279d53	2026-09-10 19:16	Kaushik192006	Update README
559ce30	2026-09-06 13:37	Kaushik192006	Test Jenkins automation
55a6539	2026-09-06 13:32	Kaushik192006	Fix Jenkins pipeline BOM
853cd30	2026-09-06 13:26	Kaushik192006	Fix Jenkins pipeline
af91a41	2026-09-06 12:51	Kaushik2	Merge pull request #1 from vinjamurkaushik-web/features/stats
c81f73a	2026-09-06 12:36	vibemax07	Add task statistics
e43f4a5	2026-09-06 12:12	Kaushik192006	Initial project setup

The history shows good practice in places – for example the dark-mode work was split into six small commits with descriptive, conventional messages ("fix: apply saved theme before paint

to prevent FOUC"). It also shows weaker messages such as "New change now" and "Showing maam"; the shared context explicitly asked the team to avoid such messages in the second phase.

2.7 Pull Requests and Code Integration

A **pull request (PR)** is a GitHub feature that proposes merging one branch into another. It shows the commits and changed files, allows team members to comment and review, and records who merged it and when. Three pull requests were merged into `main`, as listed in Table 2.4.

Table 2.4: Pull requests merged into main

PR	Source branch	Content	Merge commit	Merged on
#1	features/stats	Task statistics (total / completed / pending)	af91a41	6 Sep 2026
#2	feature/dark-mode	Dark/light theme toggle with persistence	b9f1bd6	10 Sep 2026
#3	features/stats	Task categories ("add tasks categories")	c1b3f7d	11 Sep 2026

Figure 2.4 shows pull request #3. It records that the account vibemax07 merged one commit (1452f30, "add tasks categories") from `features/stats` into `main` on 11 September, changing three files with 34 added lines.

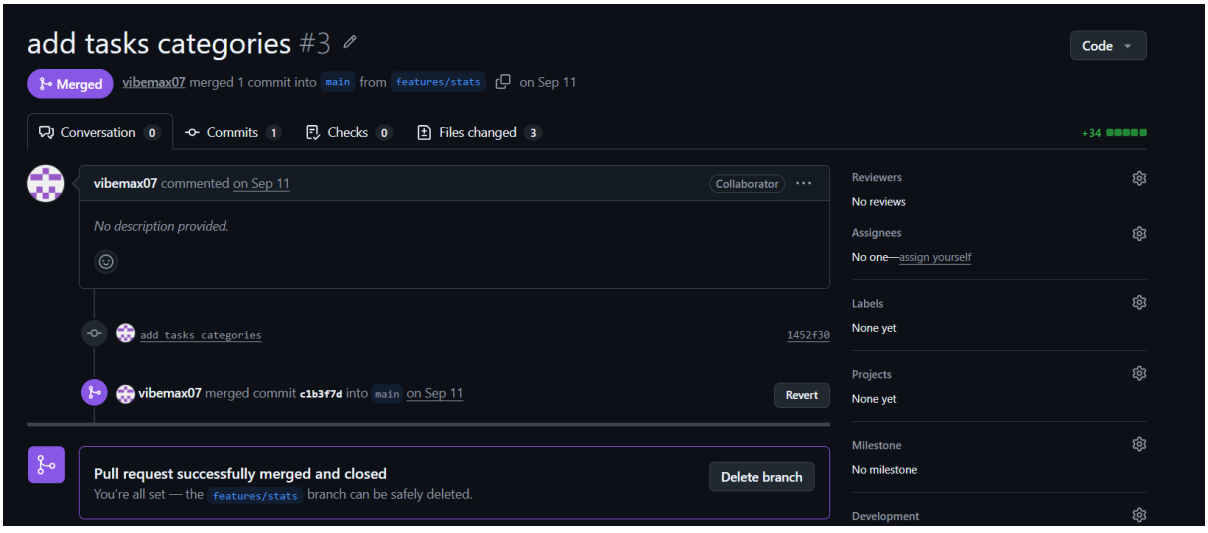


Figure 2.4: Merged pull request #3 (add tasks categories) on GitHub

2.8 Merge Workflow in the Second Phase

The shared context planned the second-phase merge order as: merge `feature/auth` into `main` first, then rebase `feature/docker-selenium` on the updated `main`, re-run the tests and merge it. In practice, because both branches were completed on the evening of 1 November 2026, the team created an integration branch, `feature/auth-selenium-integration`, from `feature/docker-selenium` and merged `feature/auth` into it with a merge commit (ef9e7a6, "Merge auth and Selenium branches"). The Selenium suite was then run against this integrated code (Chapter 6).

At the time the evidence was captured, the integration branch was **five commits ahead of main and had not yet been merged through a pull request** (Figure 2.2 shows no PR against it). The recommended next step is to open a pull request from `feature/auth-selenium-integration` to `main`, have the other members review it, and merge it, so that `main` – and therefore GitHub Pages and the Jenkins job – reflect the final application.

CHAPTER 3: APPLICATION DEVELOPMENT

3.1 Application Architecture

Student Task Manager is a **purely client-side** web application. There is no application server, no server-side code and no database server. A web server (GitHub Pages or the Nginx container) only delivers static files – HTML, CSS and JavaScript – to the browser. All behaviour then runs inside the browser, and all data is stored in the browser's **localStorage**, a simple key–value store provided by the Web Storage API. localStorage is scoped to the origin (scheme, host and port) and lives on the user's own device; it is not shared between browsers or computers and is not a database server.

Figure 3.1 shows the architecture. Each HTML page loads `auth.js`; the task dashboard additionally loads `script.js`. Both scripts read and write localStorage.

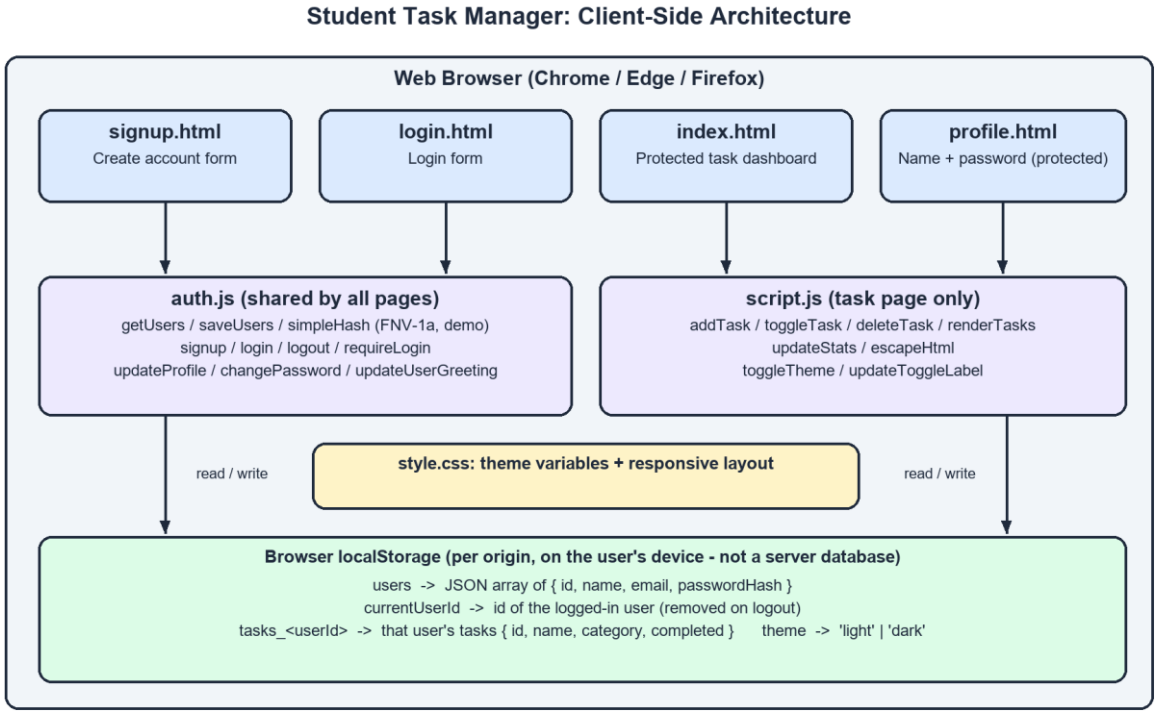


Figure 3.1: Client-side architecture of Student Task Manager

Table 3.1 summarises the role of each source file. Line counts are taken from the final integration branch.

Table 3.1: Application source files and their roles

File	Lines	Role
index.html	91	Task dashboard. Applies saved theme before paint, loads auth.js and calls requireLogin(); contains task input, category selector, statistics cards and task list
login.html	41	Login form (login-email, login-password, login-submit, login-error) and link to signup
signup.html	50	Registration form (name, email, password, confirmation) with error and success messages
profile.html	64	Protected page with profile form (name, read-only email) and change-password form
auth.js	281	Shared authentication logic: user storage, demo hash, signup, login, logout, route protection, profile update, password change, greeting
script.js	219	Task operations, statistics, per-user persistence, HTML escaping, theme toggle
style.css	522	CSS custom properties for light and dark themes, card layout, forms, responsive rules

3.2 Frontend Implementation

3.2.1 HTML Structure

All pages use semantic HTML5 elements (`header`, `main`, `section`, `form`, `footer`) and a responsive viewport meta tag. Every interactive element has a stable `id` attribute; these IDs are part of the team contract because the Selenium tests locate elements by ID. Forms use the `novalidate` attribute so that the browser's built-in validation does not pre-empt the application's own messages, which the tests check word for word. Error and success messages are rendered in paragraphs with `role="alert"` or `role="status"` so that assistive technologies announce them.

The task dashboard (`index.html`) contains a header with the title, a personalised greeting (`user-greeting`), a Profile link (`profile-link`), a Log out button (`logout-btn`) and the theme toggle (`themeToggleBtn`). Below the header are the task input section (`taskInput`, `categoryInput`, `addTaskBtn`), the statistics section (`totalTasks`, `completedTasks`, `pendingTasks`) and the task list (`taskList`, `emptyMessage`).

3.2.2 CSS Styling

`style.css` defines all colours as **CSS custom properties** on the `:root` selector – for example `--bg-page`, `--bg-card`, `--color-primary` and `--color-text`. A second rule, `[data-`

`theme="dark"]`, redefines the same variables with dark-theme values. Because every component refers to the variables rather than to fixed colours, switching the theme only requires adding or removing the `data-theme` attribute on the root `<html>` element. Transitions make the change smooth. A `@media (max-width: 480px)` rule rearranges the layout for small screens, so the application is usable on mobile phones.

Listing 3.1: Theme variables in style.css (excerpt)

```
:root {
  --bg-page:       #f0f4f8;
  --bg-card:       #ffffff;
  --color-primary: #2563eb;
  --color-text:    #333333;
}
[data-theme="dark"] {
  --bg-page:       #0f172a;
  --bg-card:       #1e293b;
  --color-primary: #60a5fa;
  --color-text:    #e2e8f0;
}
```

3.2.3 JavaScript Functionality

The application uses plain (vanilla) JavaScript without frameworks or build tools. `script.js` keeps the current user's tasks in an in-memory array, renders them into the DOM, and saves the array to localStorage after every change. `auth.js` provides functions that every page can call, and attaches its event listeners when the `DOMContentLoaded` event fires, so that each page only activates the forms it actually contains.

3.3 Task Management Operations

A task is represented as a JavaScript object `{ id, name, category, completed }`, where `id` is the timestamp returned by `Date.now()`. The main operations are:

- **addTask()** – reads and trims the task name; rejects an empty name with an alert; creates a new task with the selected category and `completed: false`; saves and re-renders. Pressing Enter in the input field also adds a task.
- **toggleTask(id)** – flips the `completed` flag of the matching task, saves and re-renders. Completed tasks are shown with a struck-through style.
- **deleteTask(id)** – removes the matching task with `Array.filter`, saves and re-renders.
- **renderTasks()** – clears the list, updates the statistics, shows the empty-state message when there are no tasks, and otherwise creates one `<li>` per task containing a checkbox, the name, a category badge and a delete button.

- **updateStats()** – counts total, completed and pending tasks and writes them into the three statistics cards.
- **escapeHtml(text)** – replaces `&`, `<`, `>`, `"` and `'` with HTML entities before user input is inserted with `innerHTML`, preventing script injection through task names.

Figure 3.2 shows the dashboard immediately after login, before any tasks have been added, and Figure 3.3 shows a task being entered with its category. The dashboard with tasks is shown in Figures 3.7 and 3.8 (Section 3.7).

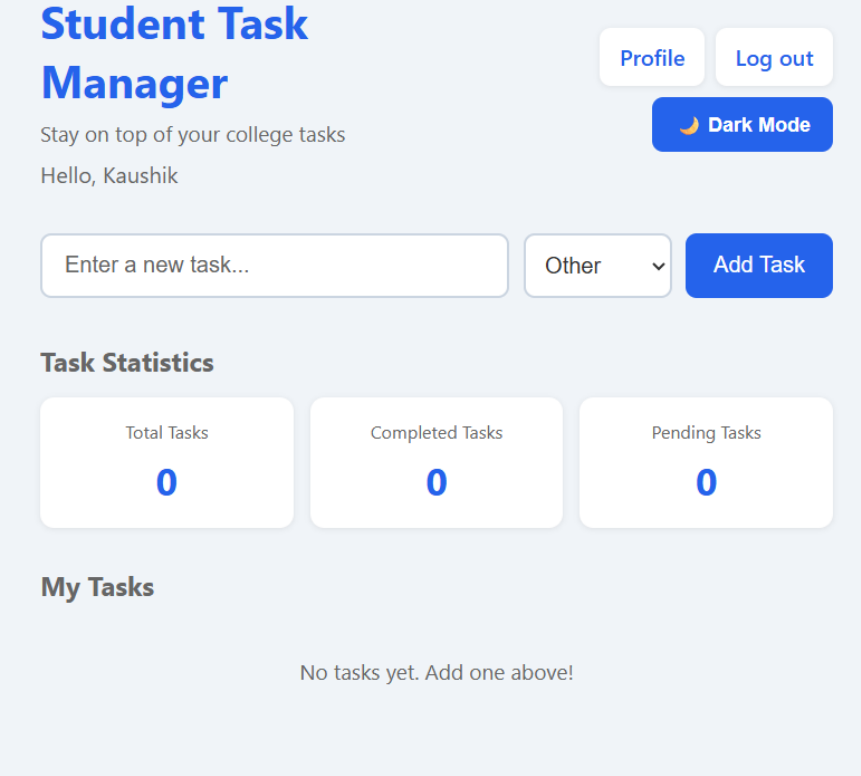


Figure 3.2: Task dashboard after login, with personalised greeting and empty task list

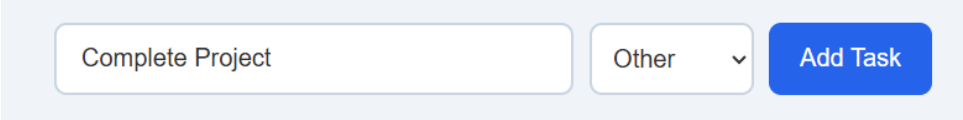


Figure 3.3: Adding a new task with a category

3.4 localStorage Persistence

The Web Storage API stores strings only. Tasks and users are therefore serialised with `JSON.stringify` before saving and parsed with `JSON.parse` when loading. Loading code is defensive: it wraps parsing in `try...catch`, checks that the result is an array, and filters out malformed entries, so that corrupted storage cannot crash the page. Table 3.2 lists the keys used, as defined in the team contract and confirmed in the source code.

Table 3.2: localStorage keys used by the application

Key	Written by	Content
users	auth.js	JSON array of { id, name, email, passwordHash }; email stored in lower case
currentUserId	auth.js	id of the logged-in user; removed on logout
tasks_<userId>	script.js	JSON array of that user's tasks { id, name, category, completed }
theme	script.js	'light' or 'dark'
studentTheme	script.js	Legacy theme key from the first phase, kept in sync for compatibility
studentUsers, studentCurrentUserId	auth.js (read only)	Legacy keys migrated to users/currentUserId if found

Listing 3.2: Per-user task persistence in script.js

```
function saveTasksToStorage(tasks) {
  if (!currentUser) return;
  try {
    localStorage.setItem('tasks_' + currentUser.id, JSON.stringify(tasks));
  } catch (error) {
    console.error('Unable to save tasks.', error);
  }
}
```

3.5 Authentication Implementation

Authentication was added on the `feature/auth` branch (commit e03b8f1), which created `auth.js`, `login.html`, `signup.html` and `profile.html` and modified `index.html`, `script.js`, `style.css` and the README – 700 inserted lines across eight files. All authentication logic runs in the browser.

3.5.1 Password Hashing (Demonstration Level)

Passwords are never stored as entered; instead `simpleHash()` computes a 32-bit FNV-1a hash and stores it with a `demo-fnv1a-` prefix. The shared context records why a stronger browser API was not used: the Web Crypto API (`crypto.subtle`) is only available in secure contexts (HTTPS or `localhost`), whereas the Selenium browser reaches the application at `http://web`, which is not a secure context. Using `crypto.subtle` would therefore have broken login during automated testing.

Listing 3.3: Demonstration hash function in auth.js

```
// This hash is deterministic but not cryptographically secure. Real applications
// must authenticate on a backend and use server-side password hashing.
function simpleHash(text) {
  let hash = 2166136261;
  for (let index = 0; index < text.length; index += 1) {
    hash ^= text.charCodeAt(index);
    hash = Math.imul(hash, 16777619);
  }
  return 'demo-fnv1a-' + (hash >>> 0).toString(16).padStart(8, '0');
}
```

FNV-1a is fast, unsalted and only 32 bits wide, so it offers **no real protection**: anyone with access to the browser can read localStorage, and short passwords can be recovered by brute force. This limitation is deliberate, is documented in the source code and the README, and is repeated in Chapter 9.

3.5.2 User Registration

`signup()` runs when the signup form is submitted. It trims the name, normalises the email to lower case, and validates the input in a fixed order. The first failing rule displays its message in the `signup-error` element and stops processing. On success, a new user with a generated id (`user-` + base-36 timestamp + random suffix) is appended to `users`, the message "Account created" is shown, and after one second the browser is redirected to `login.html`. Table 3.3 lists the validation rules and the exact messages, which match the team contract.

Table 3.3: Validation rules and messages (from the contract, verified in auth.js)

Rule	Message displayed	Where
Any required field empty	All fields are required	Signup, login, profile, password
Email not of the form name@domain.tld	Enter a valid email address	Signup, login
Password shorter than 6 characters	Password must be at least 6 characters	Signup, change password
Confirmation does not match	Passwords do not match	Signup
Email already registered (case-insensitive)	Email already registered	Signup
Unknown email or wrong password	Invalid email or password	Login
Profile saved	Profile updated	Profile
Password changed	Password changed	Profile
Current password wrong	Current password is incorrect	Profile

[EVIDENCE PLACEHOLDER]
Signup page (signup.html) – no screenshot of this page was found in the screenshots folder.
Insert the screenshot here before printing (see Report_Notes.md).

Figure 3.4: Signup page (screenshot not available in the evidence folder)

3.5.3 Login and Logout

`login()` normalises the email, checks that both fields are filled and that the email is well-formed, and then searches `users` for an entry whose email and password hash both match. The same message, "Invalid email or password", is shown whether the email is unknown or the password is wrong, so the form does not reveal which accounts exist. On success, the user's id is stored under `currentUserId` and the browser navigates to `index.html`. `logout()` removes the session keys and replaces the current page with `login.html`. Figure 3.5 shows the login page (dark theme).

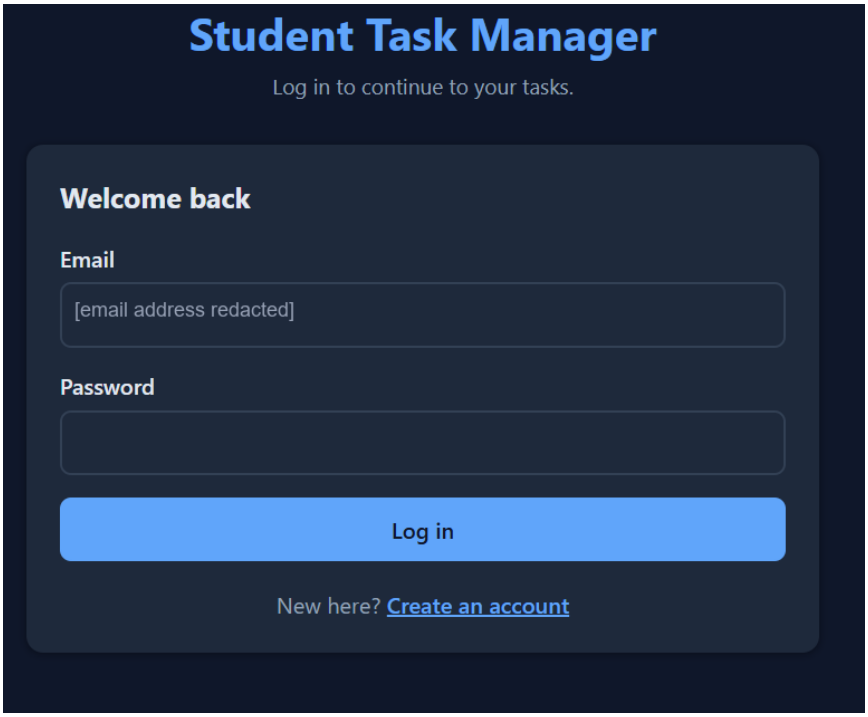


Figure 3.5: Login page in dark theme (email address redacted)

3.5.4 Protected Pages

`index.html` and `profile.html` call `requireLogin()` from an inline script in the `<head>`, before the page body is rendered. If no valid session is found, `window.location.replace('login.html')` sends the user to the login page; `replace` is used so that the protected page is not left in the browser history. Test cases TC10 and TC11 verify this behaviour.

Listing 3.4: Route protection in auth.js

```
function requireLogin() {
  if (!getCurrentUser()) {
    window.location.replace('login.html');
    return false;
  }
  return true;
}
```

This protection is a user-interface convenience rather than a security boundary: because the HTML files are static, anyone can still download them, and a user can edit localStorage in the browser's developer tools.

3.6 Profile Management and Password Change

When `profile.html` loads, the profile form is pre-filled with the current user's name and email; the email field is read-only. `updateProfile()` validates that the name is not empty, updates the user record, shows "Profile updated" and refreshes the greeting. `changePassword()` requires both fields, verifies the current password by comparing hashes ("Current password is incorrect" otherwise), enforces the six-character minimum, stores the new hash, clears the form and shows "Password changed". After a password change the old password no longer works and the new one does, which is verified by test case TC13. Figure 3.6 shows the profile page.

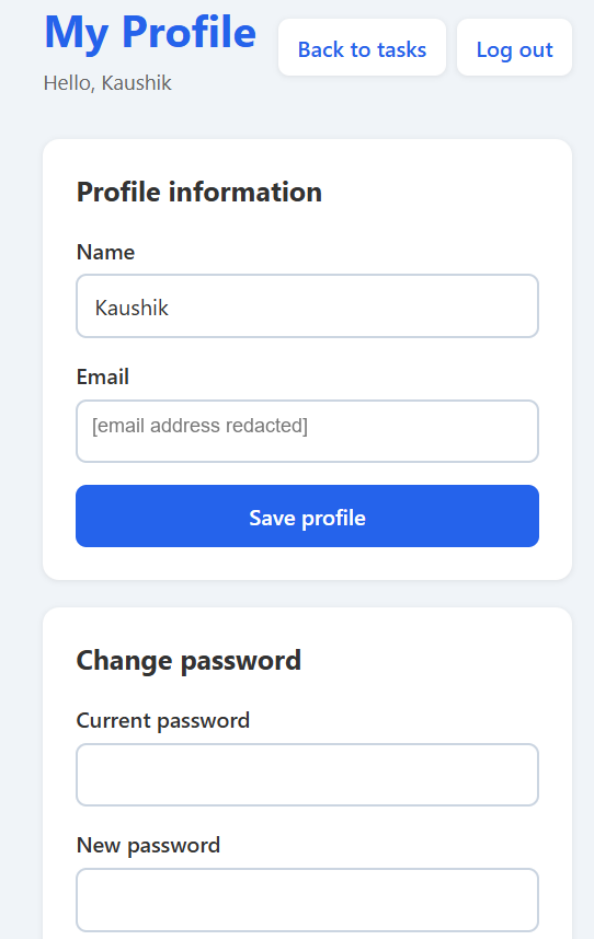


Figure 3.6: Profile page with name, read-only email (redacted) and change-password form

3.7 Theme Switching

The theme feature was developed on `feature/dark-mode` (PR #2). To avoid a "flash of unstyled content" (FOUC) – the page briefly appearing in the light theme before switching to dark – each page contains a small inline script in the `<head>` that reads the saved preference and sets `data-theme="dark"` on the `<html>` element before the body is painted. `toggleTheme()` in `script.js` flips the attribute, saves the choice to both `theme` and the legacy `studentTheme` key, and updates the button label ("Dark Mode" or "Light Mode"). Figures 3.7 and 3.8 show the same task list in the light and dark themes.

Listing 3.5: Applying the saved theme before first paint (index.html)

```
<script>
  (function () {
    if ((localStorage.getItem('theme') || localStorage.getItem('studentTheme')) ===
'dark') {
      document.documentElement.setAttribute('data-theme', 'dark');
    }
  }());
</script>
```

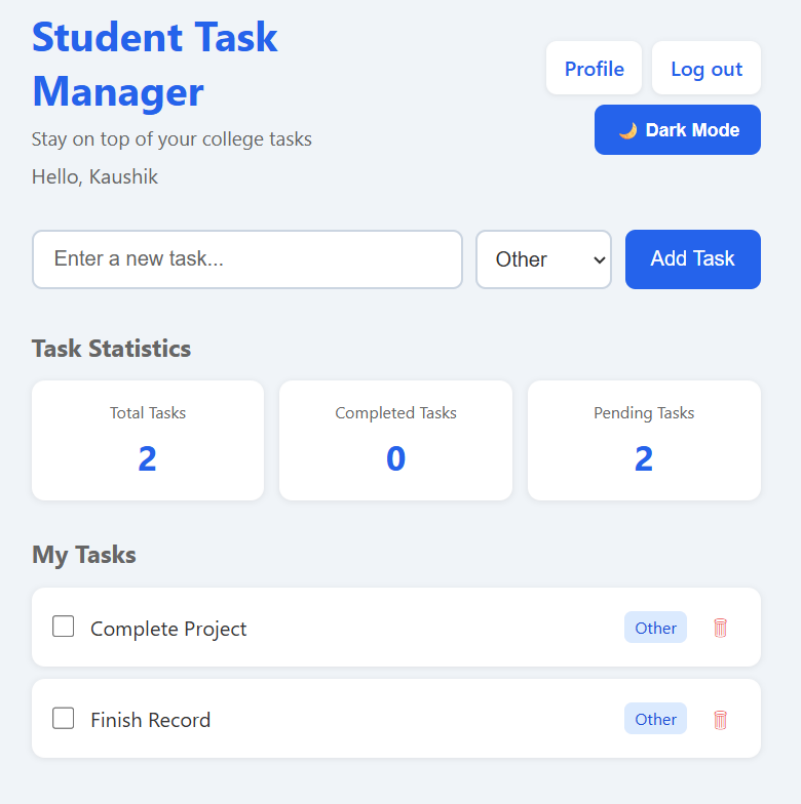


Figure 3.7: Task dashboard in light theme with two pending tasks

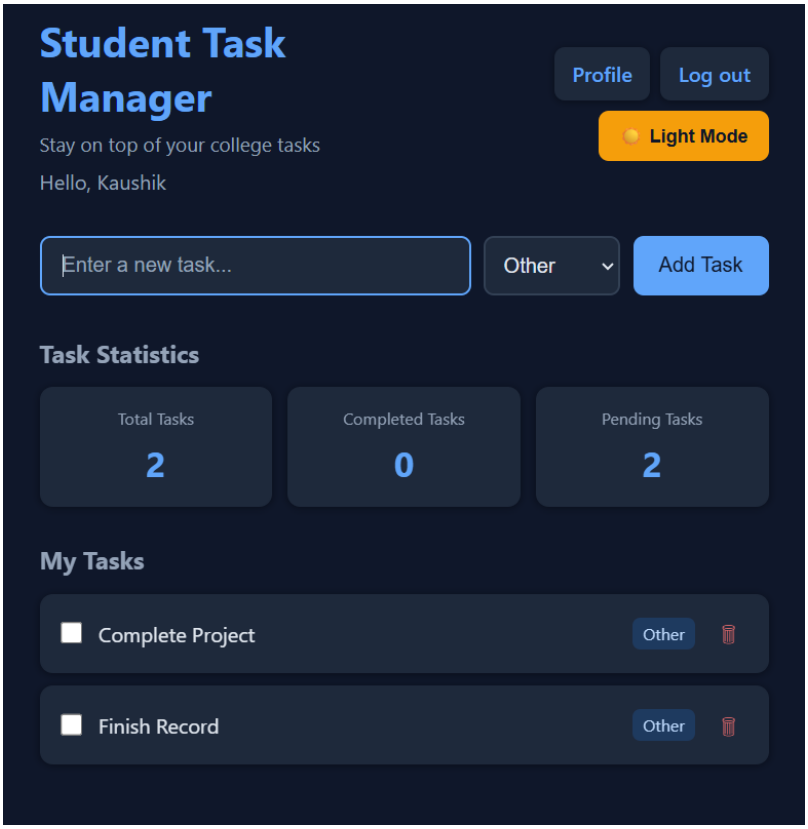


Figure 3.8: Task dashboard in dark theme with the same tasks

3.8 User-Specific Task Storage

Before authentication was added, all tasks were stored under a single key. In the final version, `script.js` first obtains the current user from `getCurrentUser()` and then reads and writes the key `tasks_<userId>`. Two accounts used in the same browser therefore have completely separate task lists, and logging out and in as another user immediately shows that user's own tasks. Test case TC15 verifies this isolation. Because the data is stored in the browser, the tasks are not available on another device or browser – a limitation addressed in the future scope.

CHAPTER 4: CONTINUOUS INTEGRATION USING JENKINS

4.1 Introduction to Continuous Integration

Continuous Integration (CI) is the practice of merging every developer's changes into a shared repository frequently and verifying each change automatically. A CI server fetches the latest code, runs a defined set of steps – such as checks, builds and tests – and reports success or failure. Problems are detected minutes after they are introduced, when they are easiest to fix, instead of weeks later during integration.

Jenkins is an open-source automation server written in Java and one of the most widely used CI tools. Jobs can be defined through the web interface or, preferably, as code in a `Jenkinsfile` committed to the repository. A Jenkinsfile written in the **declarative pipeline** syntax describes the agent on which the pipeline runs, an ordered list of stages, the steps inside each stage, and optional post-build actions.

4.2 Jenkins Installation and Configuration

Jenkins was installed on a team member's Windows computer. The workspace path shown in the console output, `C:\ProgramData\Jenkins\.jenkins\workspace\Student-Task-Manager-CI`, corresponds to the default location used by the Jenkins Windows installer when Jenkins runs as a Windows service. The general installation and configuration procedure is:

1. Install a supported Java runtime (the machine has a Microsoft OpenJDK 21 installation).
2. Run the Jenkins Windows installer, which registers Jenkins as a service listening on port 8080 by default (another port may be selected if 8080 is in use).
3. Unlock Jenkins in the browser with the initial administrator password, install the suggested plugins (which include Pipeline and Git), and create an administrator user.
4. Ensure Git is available to the Jenkins service so that it can clone the GitHub repository.

Note: the exact Jenkins version, port and plugin list were not captured in the evidence; the steps above describe the standard procedure consistent with the observed workspace path.

4.3 Jenkins Job Configuration

A Pipeline job named **Student-Task-Manager-CI** was created. The job is configured to obtain its pipeline definition from source control ("Pipeline script from SCM") using the GitHub repository URL; this is evident from the automatic **Checkout SCM** stage that appears in every build. Figure 4.1 shows the Jenkins dashboard. For Student-Task-Manager-CI, the last successful build at the time of capture was **#13**, the last failure was **#3**, and the last build took **7.7 seconds**. The other jobs listed on the dashboard (checker, gg, gugugaga, lhhhhh) have no builds and are not part of this project.

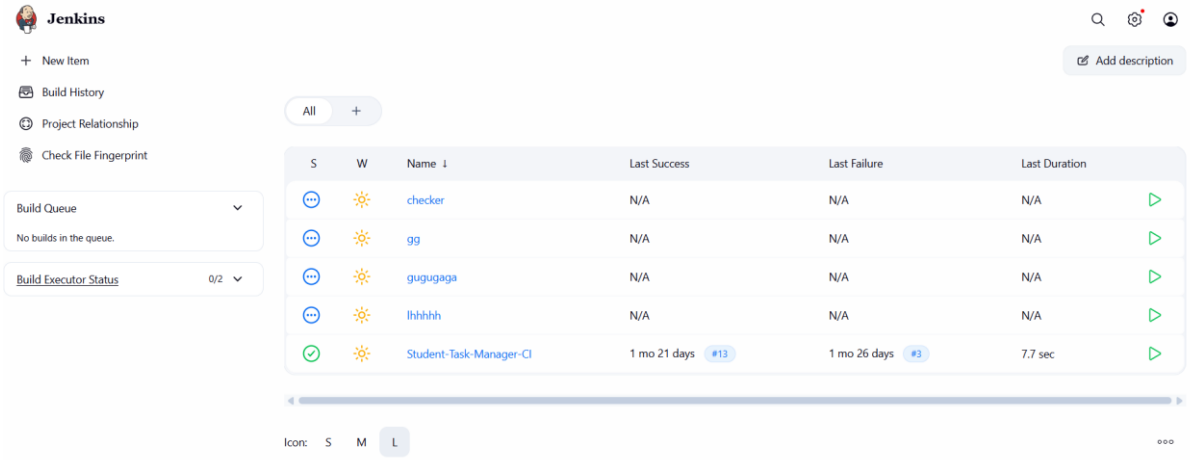


Figure 4.1: Jenkins dashboard showing the Student-Task-Manager-CI job and its last success and failure

4.4 The Jenkinsfile and Declarative Pipeline

The Jenkinsfile on the `main` branch – the version executed by the Jenkins builds shown in this report – is reproduced in full in Listing 4.1. It uses `agent any`, so it runs on any available executor, and defines three explicit stages.

Listing 4.1: Jenkinsfile on the main branch (executed in build #14)

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Validate') {
            steps {
                bat 'if not exist index.html exit /b 1'
                bat 'if not exist style.css exit /b 1'
                bat 'if not exist script.js exit /b 1'
                echo 'All required project files are present.'
            }
        }

        stage('Archive') {
            steps {
                archiveArtifacts artifacts: 'index.html,style.css,script.js,README.md',
fingerprint: true
            }
        }
    }
}
```

Figure 4.2 shows the stages in the order in which Jenkins executes them, with the durations recorded in build #14.

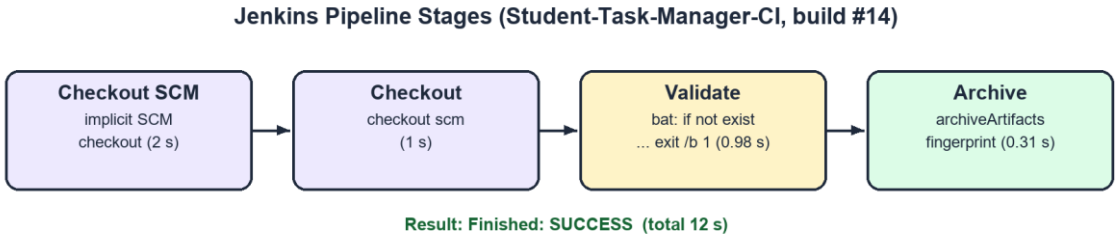


Figure 4.2: Pipeline stages executed in build #14 and their durations

4.4.1 Checkout SCM Stage

Checkout SCM is not written in the Jenkinsfile. When a declarative pipeline is loaded from source control, Jenkins automatically adds this stage to fetch the repository into the workspace before reading the Jenkinsfile's stages. In build #14 it took 2 seconds.

4.4.2 Checkout Stage

The explicit **Checkout** stage runs `checkout scm`, which checks out the same revision again. It is therefore redundant with the automatic Checkout SCM stage (1 second in build #14). It is

harmless, and it makes the checkout step visible to readers of the Jenkinsfile; it could be removed, or the automatic checkout disabled with `options { skipDefaultCheckout() }`.

4.4.3 Validate Stage

The **Validate** stage is a lightweight sanity check that the essential application files exist. Because the Jenkins agent runs on Windows, each check is a Windows batch command executed with the `bat` step. The command `if not exist index.html exit /b 1` exits with status 1 if the file is missing; Jenkins treats any non-zero exit status from `bat` as a failure, which marks the stage and the build as failed and stops the pipeline. When all files are present, the `echo` step prints "All required project files are present." In build #14 this stage took 0.98 seconds.

4.4.4 Archive Stage

The **Archive** stage uses `archiveArtifacts` to store copies of `index.html`, `style.css`, `script.js` and `README.md` with the build record, so that the exact files of any build can be downloaded later from the Jenkins interface. The option `fingerprint: true` records an MD5 checksum of each archived file; Jenkins can then show which builds produced or used a particular version of a file ("Check File Fingerprint" on the dashboard). In build #14 this stage took 0.31 seconds.

Table 4.1: Summary of pipeline stages

Stage	Defined by	Action	Fails when	Build #14
Checkout SCM	Jenkins (implicit)	Clones the repository from GitHub	Repository unreachable	2 s, success
Checkout	Jenkinsfile	checkout scm	Repository unreachable	1 s, success
Validate	Jenkinsfile	bat: if not exist <file> exit /b 1	Any required file missing	0.98 s, success
Archive	Jenkinsfile	archiveArtifacts, fingerprint: true	No matching files	0.31 s, success

4.5 Build Execution

Figure 4.3 shows build #14, started manually by Kaushik Vinjamur. All four stages completed successfully and the build took 12 seconds in total. The selected Archive stage shows the two log lines "Archiving artifacts" and "Recording fingerprints".

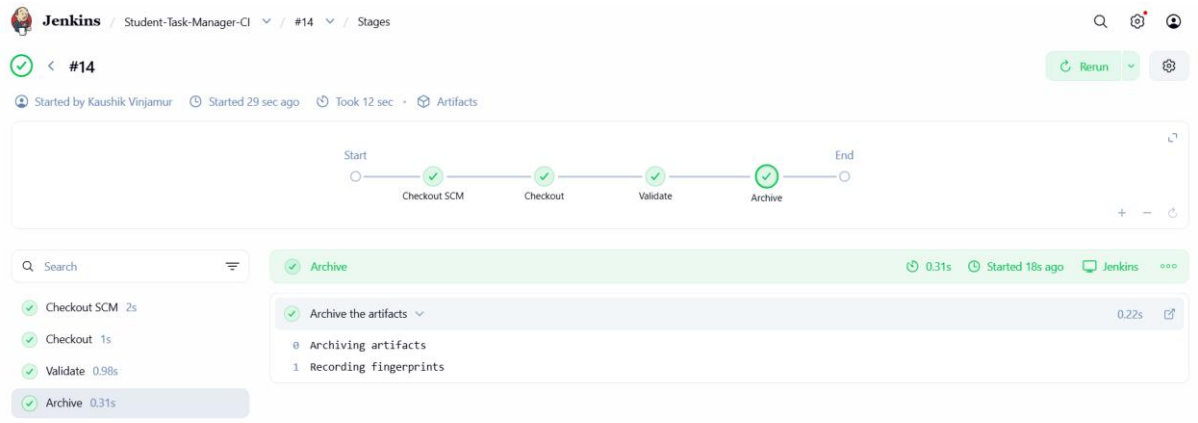


Figure 4.3: Stage view of successful build #14

4.6 Console Output

The console output records every step of a build. Figure 4.4 shows the end of the console log of build #14. The three `bat` file checks are executed in the workspace directory, the Validate stage prints its confirmation message, the Archive stage archives artifacts and records fingerprints, and the log ends with **Finished: SUCCESS**.

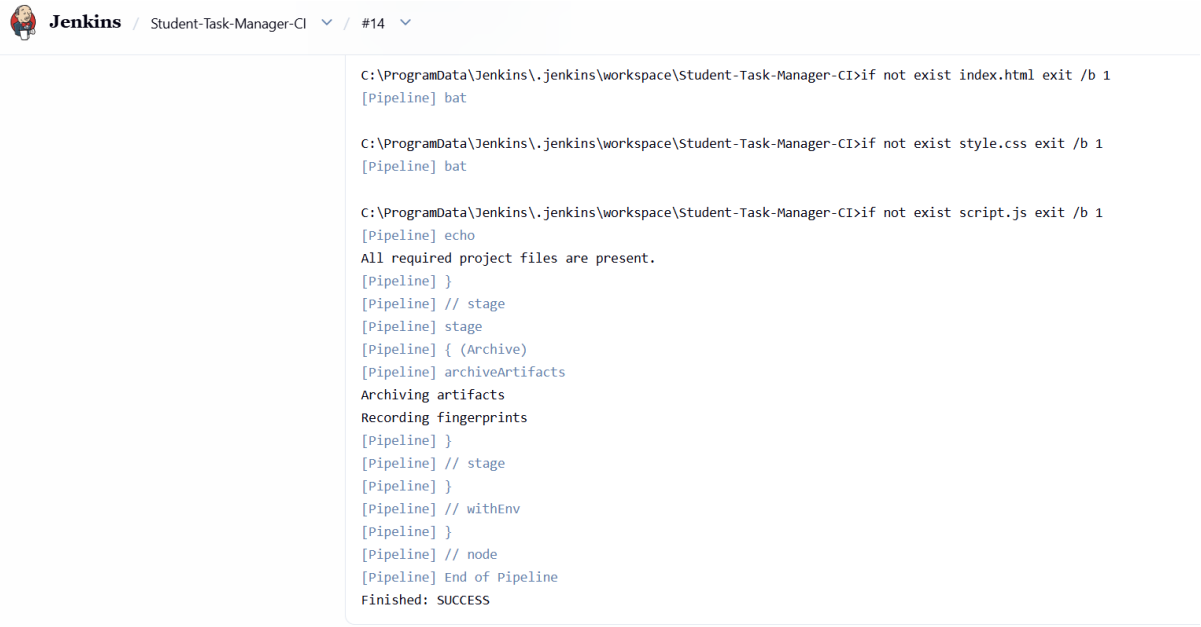


Figure 4.4: Console output of build #14 ending with Finished: SUCCESS

4.7 Build Success and Failure Handling

A Jenkins build is marked **SUCCESS** when every step returns normally. If a step fails – for example a `bat` command returns a non-zero exit code – the current stage is marked failed, later stages are skipped, and the build result is **FAILURE**. The dashboard (Figure 4.1) records the

most recent failure for this job as build **#3**. The commit history provides the context: on 6 September 2026 the commits "Fix Jenkins pipeline" (853cd30) and "Fix Jenkins pipeline BOM" (55a6539) were made shortly after the Jenkinsfile was introduced. A byte-order mark (BOM) at the start of a file saved by some Windows editors can prevent Jenkins from parsing a Jenkinsfile; removing it fixed the pipeline, and every later build shown on the dashboard succeeded. The subsequent commit "Test Jenkins automation" (559ce30) and the README test lines were used to exercise the job.

4.8 Extended Jenkinsfile on the Integration Branch

As part of the second phase, the Jenkinsfile on `feature/docker-selenium` (and therefore on `feature/auth-selenium-integration`) was extended. Its Validate stage now checks all 17 application, Docker and test files; Archive includes the new files; and two new stages and a post-build action were added (Listing 4.2). Figure 4.5 shows the beginning of this extended file in the editor.

Listing 4.2: Stages added to the Jenkinsfile on feature/auth-selenium-integration

```
        stage('Docker Build') {
            steps {
                bat 'docker compose build web'
            }
        }

        stage('Selenium Tests') {
            steps {
                bat 'docker compose --profile test up --build --abort-on-container-exit
--exit-code-from tests'
            }
        }
    }

    post {
        always {
            bat 'docker compose down'
        }
    }
```

Jenkinsfile

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Validate') {
            steps {
                bat 'if not exist index.html exit /b 1'
                bat 'if not exist login.html exit /b 1'
                bat 'if not exist signup.html exit /b 1'
                bat 'if not exist profile.html exit /b 1'
                bat 'if not exist style.css exit /b 1'
                bat 'if not exist script.js exit /b 1'
                bat 'if not exist auth.js exit /b 1'
                bat 'if not exist Dockerfile exit /b 1'
                bat 'if not exist .dockerignore exit /b 1'
                bat 'if not exist docker-compose.yml exit /b 1'
```

Figure 4.5: Extended Jenkinsfile on the integration branch (Validate stage excerpt)

Important distinction: the Jenkins job builds the `main` branch, and the extended Jenkinsfile has not yet been merged into `main`. The console output of build #14 (Figure 4.4) confirms that only `index.html`, `style.css` and `script.js` were validated – exactly the three checks of the main-branch Jenkinsfile. **No Jenkins build in the available evidence executed the Docker Build or Selenium Tests stages.** These stages also require Docker Engine and the Compose plugin to be accessible to the Jenkins Windows service, which has not been demonstrated. Table 4.2 compares the two versions.

Table 4.2: Executed Jenkinsfile (main) compared with the extended Jenkinsfile

Aspect	main (executed, build #14)	feature/auth-selenium-integration (not yet executed)
Stages	Checkout, Validate, Archive	Checkout, Validate, Archive, Docker Build, Selenium Tests
Files validated	3 (index.html, style.css, script.js)	17 (pages, scripts, Docker files, test files)
Archived files	4	Application, Docker and test files, README, Jenkinsfile
Docker / Selenium	None	docker compose build; compose test profile with --exit-code-from tests
Post actions	None	post { always { docker compose down } }
Evidence	Builds #13 and #14 successful	No Jenkins run captured

4.9 Jenkins CI and the Selenium Test Suite

In this project, Jenkins CI and the Selenium suite are **separate** activities. Jenkins performs structural validation and archiving of the repository on every build. The Selenium suite (Chapter 6) was executed manually by a team member on the local machine with Docker Compose. The extended Jenkinsfile is designed to bring the two together, so that every push would also build the image and run the browser tests; activating it is listed as future work in Chapter 9.

CHAPTER 5: CONTAINERISATION USING DOCKER

5.1 Introduction to Containerisation

Containerisation packages an application together with everything it needs to run – runtime, libraries, configuration and files – into a single, portable unit called a **container**. Containers share the host operating system's kernel but run in isolated user spaces, so they start in seconds and use far fewer resources than virtual machines. A container is created from an **image**, a read-only template built in layers from instructions in a `Dockerfile`. Images can be stored in registries such as Docker Hub and run identically on any machine with a container engine.

5.2 Why Docker Was Used

- **Consistency:** the application is served by the same Nginx version on every team member's computer, removing "it works on my machine" problems.
- **Realistic serving:** opening `index.html` directly from disk uses the `file://` scheme, which behaves differently from a real website. Nginx serves the files over HTTP, as a real deployment would.
- **Reproducible test environment:** Selenium needs a browser, a matching driver and a reachable application. Docker Compose provides all three on one private network with a single command.
- **Infrastructure as code:** the `Dockerfile` and `docker-compose.yml` are committed to Git, so the environment is versioned and reviewed like the application code.

5.3 Dockerfile Explanation

Listing 5.1 shows the complete `Dockerfile` for the web application.

Listing 5.1: Dockerfile for the web application

```
# Use the small official Nginx image to serve this static website.
FROM nginx:alpine

# Copy every page and browser-side file into Nginx's public web directory.
COPY index.html login.html signup.html profile.html style.css script.js auth.js
/usr/share/nginx/html/

# Document that the container serves HTTP on port 80.
EXPOSE 80
```

- **FROM nginx:alpine** – uses the official Nginx web-server image built on Alpine Linux, which is very small. Nginx is ideal for static sites: it is fast, needs no configuration for this use case, and its default command starts the server automatically.
- **COPY ... /usr/share/nginx/html/** – copies exactly the seven files that make up the site into Nginx's default document root. Listing the files explicitly ensures that nothing else (tests, documentation, Git metadata) ends up in the image.
- **EXPOSE 80** – documents that the container listens on port 80. It does not publish the port by itself; publishing is done by Docker Compose (Section 5.4).

The `.dockerignore` file further excludes `.git`, `.gitignore`, the `tests` directory, `docker.md` and `00 shared context.md` from the build context, which keeps builds fast and avoids sending unnecessary files to the Docker engine.

5.4 Port Mapping

Inside the container Nginx listens on port 80. The Compose file publishes it as `"8080:80"`, meaning host port **8080** is forwarded to container port **80**. The application is therefore opened in the browser at `http://localhost:8080`. Port 8080 was chosen to avoid needing administrator rights for port 80 and to avoid conflicts with other local services. (Jenkins also defaults to port 8080, so the two cannot both use it at the same time on one machine; one of them must be moved to another port.)

5.5 Docker Image Creation and Container Execution

The image can be built and run either directly with the Docker CLI or through Docker Compose, which the team used. Table 5.1 lists the commands documented in the README and used during the project.

Table 5.1: Docker and Docker Compose commands used

Command	Purpose
docker compose up --build	Build the web image (if needed) and start the web service; app at http://localhost:8080
docker compose build web	Build only the web image, tagged sohails2409/student-task-manager:latest
docker compose push web	Publish the web image to Docker Hub (requires docker login)
docker compose --profile test run --build --rm tests	Start web and Selenium, build the test image, run the Selenium suite, remove the test container

Command	Purpose
docker ps	List running containers, their status, health and port mappings
docker compose down	Stop and remove the project's containers and network

Equivalent plain Docker commands for the web image alone would be `docker build -t student-task-manager .` followed by `docker run -d -p 8080:80 student-task-manager`.

5.6 Docker Compose

Docker Compose defines and runs multi-container applications from a single YAML file. The project's `docker-compose.yml` defines three services. Figure 5.1 shows the file as committed, and Figure 5.2 illustrates how the services relate to each other and to the host.

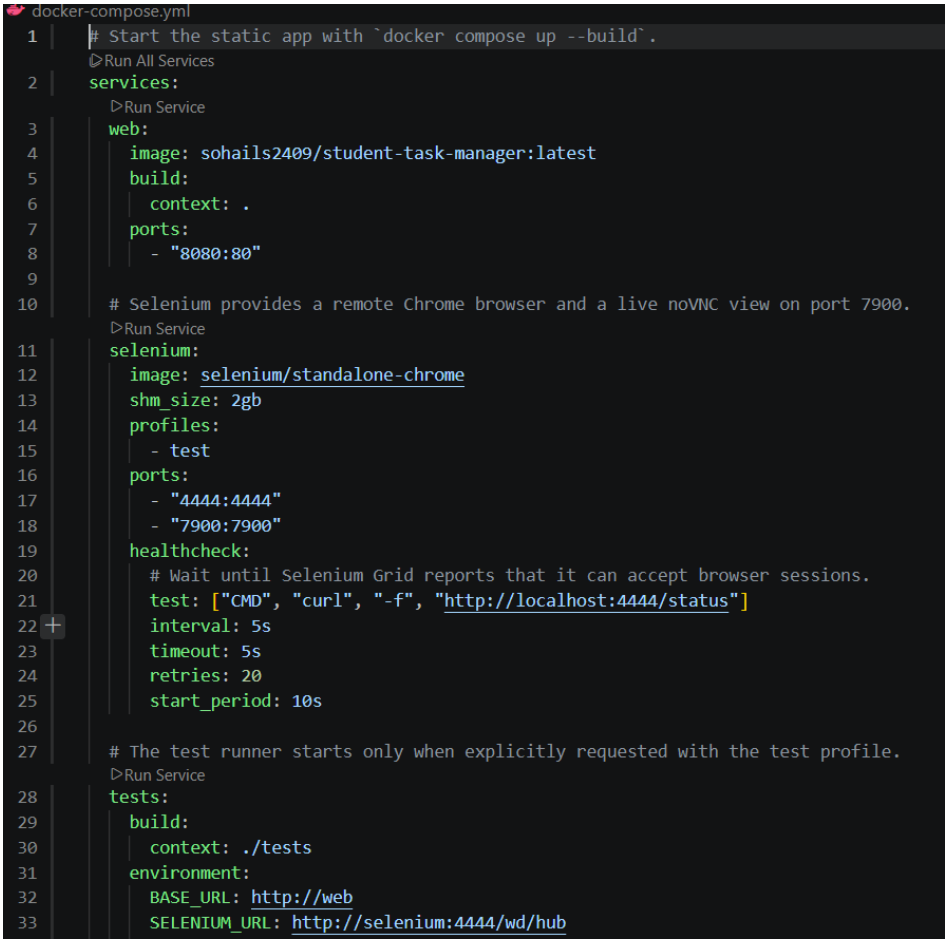


Figure 5.1: docker-compose.yml with the web, selenium and tests services

Table 5.2: Docker Compose services (verified from docker-compose.yml)

Service	Image / build	Ports	Profile	Key settings
web	build: . (Dockerfile); tagged sohails2409/student-task-manager:latest	8080:80	none (always starts)	Serves the static site with Nginx
selenium	selenium/standalone-chrome	4444:4444, 7900:7900	test	shm_size: 2gb; healthcheck curl -f http://localhost:4444/status every 5 s, 20 retries, 10 s start period
tests	build: ./tests (python:3.12-slim)	none	test	BASE_URL=http://web; SELENIUM_URL=http://selenium:4444/wd/hub; PYTHONUNBUFFERED=1; depends_on web (started) and selenium (healthy)

5.6.1 Profiles

The `selenium` and `tests` services are assigned to the `test` **profile**. Services with a profile are started only when that profile is requested. A plain `docker compose up --build` therefore starts only the `web` service – ideal for a deployment demonstration – while `docker compose --profile test run --build --rm tests` additionally starts Selenium and runs the tests.

5.6.2 Network Communication

Compose automatically creates a private bridge network for the project (here named after the project directory, `devop_project`) and attaches every service to it. On this network each service is reachable by its **service name**, resolved by Docker's internal DNS. The test runner therefore connects to the browser at `http://selenium:4444/wd/hub`, and the Chrome browser inside the `selenium` container loads the application from `http://web` (port 80 inside the network). The host only needs the published ports: 8080 for the application, 4444 for the Selenium Grid and 7900 for the noVNC live view of the browser.

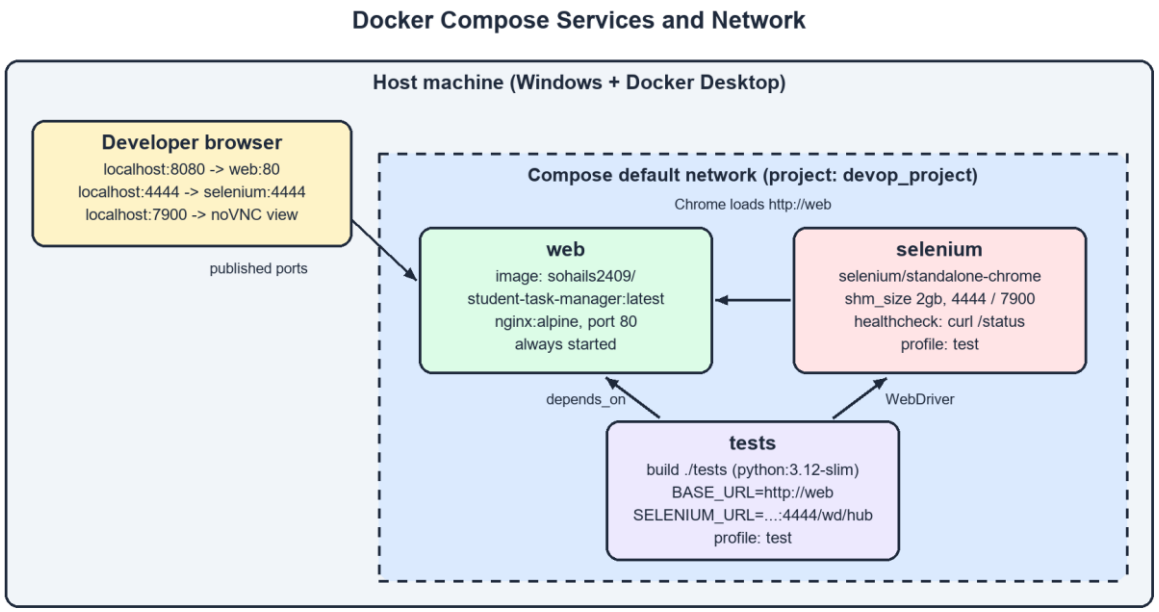


Figure 5.2: Docker Compose services, network and published ports

5.6.3 Health Checks and Start-up Order

A container being "started" does not mean the program inside is ready. The Selenium Grid takes several seconds to initialise, and tests that connect too early fail. The `selenium` service therefore defines a **health check** that runs `curl -f http://localhost:4444/status` inside the container every 5 seconds (5-second timeout, up to 20 retries, after a 10-second start period). The `tests` service declares `depends_on: selenium: condition: service_healthy`, so Compose starts the tests only after the Grid reports healthy. The web service only needs to be started (`condition: service_started`), because Nginx is ready almost instantly.

5.7 Running Containers

Figure 5.3 shows the output of `docker ps` during the test session. Two containers are running: `devop_project-selenium-1` from the `selenium/standalone-chrome` image, "Up 3 minutes (healthy)" with ports 4444 and 7900 published; and `devop_project-web-1`, "Up 12 minutes" with host port 8080 mapped to container port 80. The test-runner container is not listed because it runs only for the duration of the test run and is removed afterwards (`--rm`).

```
CONTAINER ID   IMAGE                      COMMAND                  CREATED          STATUS                    PORTS
                                                                   NAMES
c72adea93c83   selenium/standalone-chrome   "/opt/bin/entry_poin…"   3 minutes ago    Up 3 minutes (healthy)    0.0.0.0:4444->4444/tc
p, [::]:4444->4444/tcp, 0.0.0.0:7900->7900/tcp, [::]:7900->7900/tcp   devop_project-selenium-1
a36c2f5d4f53   c9e43a1d0b2c               "/docker-entrypoint.…"   17 minutes ago   Up 12 minutes             0.0.0.0:8080->80/tcp,
 [::]:8080->80/tcp                                                 devop_project-web-1
PS C:\Devop_Project>
```

Figure 5.3: docker ps showing the healthy Selenium container and the web container with 8080->80

[EVIDENCE PLACEHOLDER]
Terminal output of 'docker compose up --build' showing the image build and the web container starting.
Insert the screenshot here before printing (see Report_Notes.md).

Figure 5.4: Output of docker compose up --build (screenshot not available)

5.8 Test-Runner Image

The `tests/Dockerfile` builds the Python test-runner image. It starts from `python:3.12-slim`, sets `/tests` as the working directory, installs the dependencies from `requirements.txt` (`pytest>=8.0,<9.0` and `selenium>=4.20,<5.0`) before copying the test sources (so that the dependency layer is cached between builds), and finally runs Pytest:

Listing 5.2: Test-runner command in tests/Dockerfile

```
CMD ["python", "-u", "-m", "pytest", "-vv", "-rA", "-s"]
```

The `-u` flag disables output buffering, `-vv` prints each test name and result as it finishes, `-rA` adds a summary of all outcomes at the end, and `-s` shows any output printed by the tests.

CHAPTER 6: AUTOMATED TESTING USING SELENIUM AND PYTEST

6.1 Introduction to Software Testing

Software testing is the process of executing a program with the intention of finding defects and confirming that it meets its requirements. Tests are commonly organised in levels: **unit tests** check individual functions in isolation, **integration tests** check that components work together, and **end-to-end (E2E) or system tests** exercise the complete application the way a user would. Manual testing is slow and error-prone when repeated after every change; automated tests can be re-run in seconds and are therefore a cornerstone of DevOps.

This project focuses on **automated end-to-end functional testing**. The tests drive a real Chrome browser through the application's pages, fill in forms, click buttons, and check visible messages, URLs and stored state – testing the behaviour that a user actually experiences.

6.2 Selenium WebDriver

Selenium is an open-source suite for browser automation. Its core component, **WebDriver**, is a W3C-standard API through which a program sends commands such as "open this URL", "find the element with this ID", "type this text" and "click" to a browser. In this project the browser runs inside the `selenium/standalone-chrome` container, and the tests connect to it remotely through `webdriver.Remote` at `http://selenium:4444/wd/hub`. The container also provides a noVNC server on port 7900, which allows anyone to watch the automated browser live at `http://localhost:7900`.

The tests use **explicit waits** (`WebDriverWait` with expected conditions such as `url_contains`, or custom conditions that compare an element's text) instead of fixed `sleep` calls. An explicit wait polls until the condition holds or a timeout (5 seconds here) expires. This makes the tests both faster and more reliable – important because, for example, signup redirects to the login page only after a one-second delay.

6.3 Pytest

Pytest is a Python testing framework. Tests are plain functions whose names start with `test_`, written in files whose names start with `test_`. Pytest discovers them automatically, runs them, and reports each result. Its **fixtures** provide reusable set-up and tear-down code that is injected

into tests by parameter name. The project uses two fixtures and two helper functions defined in `tests/conftest.py`:

- **base_url** – returns the application URL from the `BASE_URL` environment variable (default `http://web`).
- **browser** – creates a **new remote Chrome session for each test**, sets the window size to 1280 × 900, yields the driver to the test, and always quits it afterwards. A fresh session means fresh, empty localStorage, so tests cannot affect each other.
- **signup(...)** and **login(...)** – open the relevant page and fill in the form using the contract element IDs, avoiding duplicated code in the tests.

Listing 6.1: Remote browser fixture in tests/conftest.py

```
@pytest.fixture
def browser():
    """Create one clean remote Chrome session for each test."""
    options = webdriver.ChromeOptions()
    driver = webdriver.Remote(
        command_executor=os.getenv("SELENIUM_URL", "http://selenium:4444/wd/hub"),
        options=options,
    )
    try:
        driver.set_window_size(1280, 900)
        yield driver
    finally:
        driver.quit()
```

6.4 Dockerised Testing Environment

The complete test environment is defined in Docker Compose (Chapter 5). Table 6.1 summarises the environment of the observed run; the Python and Pytest versions are taken from the session header in Figure 6.1.

Table 6.1: Test environment

Component	Value
Application under test	web service (nginx:alpine) at http://web inside the Compose network
Browser	Google Chrome in selenium/standalone-chrome, WebDriver at http://selenium:4444/wd/hub
Test runner	python:3.12-slim container; Python 3.12.16, pytest 8.4.2, pluggy 1.7.0 (platform linux)
Libraries	selenium>=4.20,<5.0; pytest>=8.0,<9.0
Command	docker compose --profile test run --build --rm tests
Code version	feature/auth-selenium-integration (auth merged with Docker/Selenium work)

6.5 Test Execution Workflow

When the command `docker compose --profile test run --build --rm tests` is executed, the following happens:

1. Compose builds the test-runner image from `./tests` (and the web image if necessary).
2. Compose starts the `web` service and the `selenium` service on the project network.
3. Compose waits until the Selenium health check (`curl -f http://localhost:4444/status`) succeeds.
4. The `tests` container starts and runs `python -u -m pytest -vv -rA -s`, which collects the 16 tests.
5. For each test, the `browser` fixture opens a new Chrome session in the Selenium container; the test drives Chrome to pages under `http://web`; the session is closed when the test ends.
6. Pytest prints each result, a summary of all outcomes, and the total count and duration. The container exits with status 0 if all tests pass, and is then removed (`--rm`).

6.6 Test Case Design

The test cases were designed from the team contract **before** the authentication code was written, so that the contract – not the implementation – defined the expected behaviour. Each validation rule and each user flow has at least one test. Tests use exact message strings, so any change to a message would be detected. The cases fall into four groups:

- **Signup (TC01–TC06):** one positive case and five negative cases, one for each validation rule.
- **Login and access control (TC07–TC11):** valid login, wrong password, unknown email, direct access while logged out, and logout followed by an attempt to reopen the task page.
- **Profile (TC12–TC13):** name update reflected in the greeting, and password change followed by logout and login with the new password.
- **Tasks and theme (TC14–TC16):** task persistence across refresh, task isolation between two users, and theme persistence across refresh.

Listing 6.2 shows a representative test. TC15 creates a first user, adds a task, logs out, creates a second user, checks that the first user's task is not visible, adds a task, logs out, logs back in as the first user and checks that only the first user's task is visible.

Listing 6.2: Test case TC15 (tests/test_tasks.py, lightly reformatted)

```
def test_tc15_two_users_have_separate_task_lists(browser, base_url):
    create_and_login(browser, base_url, "First Student",
                     "tc15-first@example.com", "first-password")
    add_task(browser, "First user's private task")
    browser.find_element(By.ID, "logout-btn").click()
    WebDriverWait(browser, 5).until(EC.url_contains("/login.html"))

    create_and_login(browser, base_url, "Second Student",
                     "tc15-second@example.com", "second-password")
    assert "First user's private task" not in task_names(browser)
    add_task(browser, "Second user's private task")
    browser.find_element(By.ID, "logout-btn").click()
    WebDriverWait(browser, 5).until(EC.url_contains("/login.html"))

    login(browser, base_url, "tc15-first@example.com", "first-password")
    WebDriverWait(browser, 5).until(EC.url_contains("/index.html"))
    names = task_names(browser)
    assert "First user's private task" in names
    assert "Second user's private task" not in names
```

6.7 Test Results

The suite was executed once with the command shown in Table 6.1. Figure 6.1 shows the start of the session: Pytest collected **16 items** and reported each as PASSED. Figure 6.2 shows the short test summary and the final line: **16 passed in 34.01 s**.

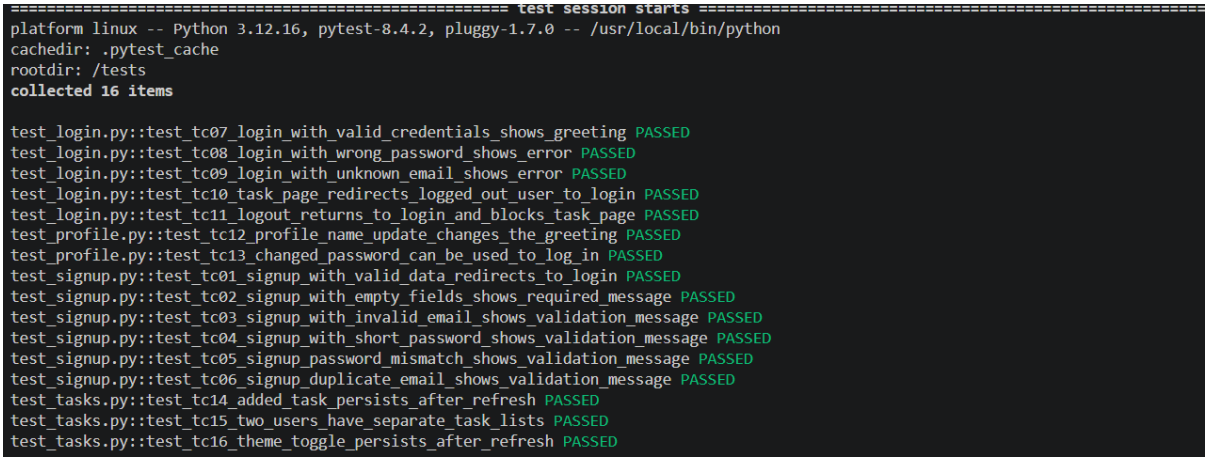


Figure 6.1: Pytest session header and per-test results (16 items collected)

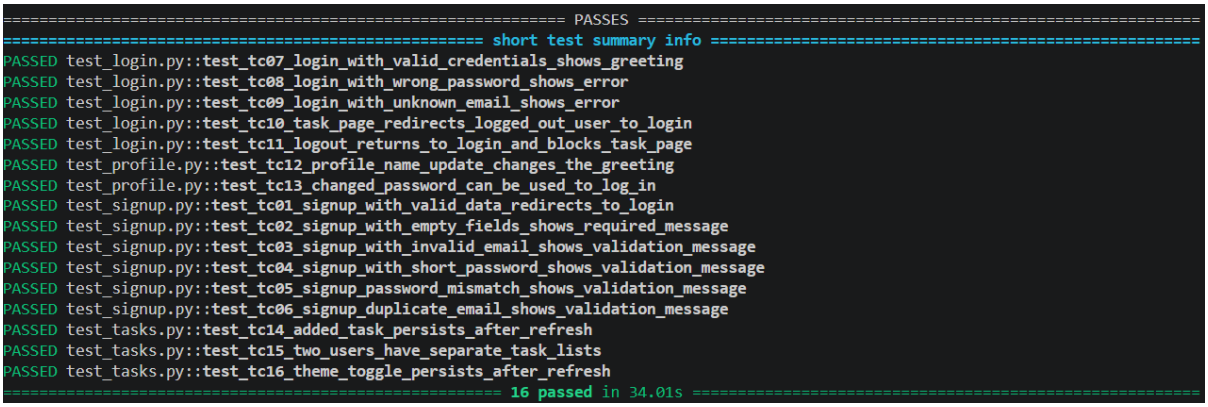


Figure 6.2: Short test summary showing 16 passed in 34.01 seconds

Table 6.2 records each test case with its expected and actual result. The actual results are taken from the observed run in Figures 6.1 and 6.2.

Table 6.2: Selenium test cases and results

ID	Test description	Expected result	Actual result	Status
TC01	Sign up with valid name, email and password	Redirect to login.html	As expected	PASS
TC02	Submit the signup form with all fields empty	"All fields are required"	As expected	PASS
TC03	Sign up with email "not-an-email"	"Enter a valid email address"	As expected	PASS
TC04	Sign up with 5-character password "short"	"Password must be at least 6 characters"	As expected	PASS
TC05	Sign up with a different confirmation password	"Passwords do not match"	As expected	PASS
TC06	Register the same email twice	"Email already registered"	As expected	PASS
TC07	Log in with valid credentials	index.html opens; greeting is not empty	As expected	PASS
TC08	Log in with a wrong password	"Invalid email or password"	As expected	PASS
TC09	Log in with an unregistered email	"Invalid email or password"	As expected	PASS
TC10	Open index.html without logging in	Redirect to login.html	As expected	PASS
TC11	Log out, then open index.html again	login.html shown; task page blocked	As expected	PASS
TC12	Change the profile name and return to tasks	"Profile updated"; greeting shows new name	As expected	PASS
TC13	Change password, log out, log in with new password	"Password changed"; login succeeds	As expected	PASS
TC14	Add a task and refresh the page	Task is still listed	As expected	PASS
TC15	Two users each add a task; switch accounts	Each user sees only their own task	As expected	PASS

ID	Test description	Expected result	Actual result	Status
TC16	Toggle to dark theme and refresh	data-theme remains "dark"	As expected	PASS

Table 6.3 maps each test ID to its Pytest function name, so that the table above can be traced directly to the source code and to the terminal output.

Table 6.3: Test IDs and Pytest node identifiers

ID	Pytest node ID
TC01	test_signup.py::test_tc01_signup_with_valid_data_redirects_to_login
TC02	test_signup.py::test_tc02_signup_with_empty_fields_shows_required_message
TC03	test_signup.py::test_tc03_signup_with_invalid_email_shows_validation_message
TC04	test_signup.py::test_tc04_signup_with_short_password_shows_validation_message
TC05	test_signup.py::test_tc05_signup_password_mismatch_shows_validation_message
TC06	test_signup.py::test_tc06_signup_duplicate_email_shows_validation_message
TC07	test_login.py::test_tc07_login_with_valid_credentials_shows_greeting
TC08	test_login.py::test_tc08_login_with_wrong_password_shows_error
TC09	test_login.py::test_tc09_login_with_unknown_email_shows_error
TC10	test_login.py::test_tc10_task_page_redirects_logged_out_user_to_login
TC11	test_login.py::test_tc11_logout_returns_to_login_and_blocks_task_page
TC12	test_profile.py::test_tc12_profile_name_update_changes_the_greeting
TC13	test_profile.py::test_tc13_changed_password_can_be_used_to_log_in
TC14	test_tasks.py::test_tc14_added_task_persists_after_refresh
TC15	test_tasks.py::test_tc15_two_users_have_separate_task_lists
TC16	test_tasks.py::test_tc16_theme_toggle_persists_after_refresh

Table 6.4: Summary of results by module

Module	Test cases	Passed	Failed
test_signup.py	6 (TC01–TC06)	6	0
test_login.py	5 (TC07–TC11)	5	0
test_profile.py	2 (TC12–TC13)	2	0
test_tasks.py	3 (TC14–TC16)	3	0
Total	**16**	**16**	**0**

6.8 Analysis of Results

All 16 tests passed in the observed run, in 34.01 seconds – an average of about 2.1 seconds per test, including creating and closing a fresh Chrome session for each one. The results show that:

- every validation rule in the contract produces exactly the agreed message;
- successful signup, login and logout redirect to the correct pages, and protected pages cannot be opened without a session;
- profile changes are saved and are visible elsewhere in the application, and password changes take effect for subsequent logins;
- tasks and the theme preference persist across page reloads, and tasks are correctly isolated per user.

Because the contract was written first and both the implementation and the tests were built against it, the integration of the two independently developed branches worked as intended. The results should nevertheless be interpreted with care: they come from a single run on one machine with one browser (Chrome); the suite tests functional behaviour only and does not cover performance, accessibility, security or cross-browser compatibility; and the suite was run manually rather than by Jenkins. These points are revisited in Chapter 9.

CHAPTER 7: DEPLOYMENT AND RELEASE

7.1 Deployment Approaches Used

Two deployment approaches appear in the project. They serve different purposes and must not be confused: GitHub Pages provides public static hosting of the repository's files, whereas Docker provides a local, containerised deployment used for demonstration and testing. **No Docker container was deployed to a cloud platform.** Table 7.1 compares them.

Table 7.1: GitHub Pages deployment compared with local Docker deployment

Aspect	GitHub Pages	Local Docker deployment
Where it runs	GitHub's static hosting infrastructure	Team member's Windows computer (Docker Desktop)
Who can access it	Anyone with the public URL	The local machine at http://localhost:8080
Web server	GitHub Pages	Nginx (nginx:alpine) inside the web container
Source of files	Repository branch configured in GitHub Pages settings	Files copied into the image by the Dockerfile from the local checkout
Evidence	github-pages environment with 8 deployments (Figure 2.1)	docker ps (Figure 5.3); application screenshots (Chapter 3)
Includes authentication?	Only after the integration branch is merged into the deployed branch	Yes – built from feature/auth-selenium-integration

7.2 Application Deployment Using Docker

Deployment with Docker consists of building the web image and starting a container from it. With Docker Desktop running, a single command in the project directory performs both steps:

Listing 7.1: Deploying the application locally

```
docker compose up --build
```

Compose builds the image from the `Dockerfile` (tagging it `sohails2409/student-task-manager:latest`), creates the project network and starts the `web` container. Because the `selenium` and `tests` services belong to the `test` profile, they are not started. The command runs in the foreground and shows the Nginx logs; `Ctrl+C` stops the services, and `docker compose down` removes them. Adding `-d` runs the services in the background instead.

7.3 Nginx Hosting

Inside the container, Nginx serves the files in `/usr/share/nginx/html` on port 80. Requesting `/` returns `index.html`, the default index document. Because all application behaviour runs in the browser, no server-side configuration is needed: Nginx simply returns each requested HTML, CSS or JavaScript file with the appropriate content type. The authentication pages are ordinary static files at `/login.html`, `/signup.html` and `/profile.html`.

7.4 Port Mapping and Access Through localhost

The mapping `8080:80` forwards connections to port 8080 on the host to port 80 in the container. The `docker ps` output in Figure 5.3 confirms the mapping (`0.0.0.0:8080->80/tcp`). Opening `http://localhost:8080` in a browser therefore loads the application from the container. As the user is not yet logged in, `requireLogin()` redirects to `login.html`; after signing up and logging in the user reaches the dashboard shown in Figures 3.2, 3.7 and 3.8.

Because localStorage is scoped per origin, accounts and tasks created at `http://localhost:8080` are separate from those created when the files are opened from disk or from GitHub Pages.

[EVIDENCE PLACEHOLDER]
Browser window showing the application with 'localhost:8080' visible in the address bar.
Insert the screenshot here before printing (see Report_Notes.md).

Figure 7.1: Application opened at http://localhost:8080 with the address bar visible (screenshot not available)

7.5 GitHub Pages Deployment

GitHub Pages is a static-site hosting service built into GitHub. Because Student Task Manager consists only of static files at the repository root, it can be published without any build step. The repository overview (Figure 2.1) shows a `github-pages` deployment environment with **eight deployments**, the latest two months before the evidence was captured, and the main-branch README describes the project as demonstrating "GitHub Pages deployment". GitHub Pages redeploys automatically when the configured branch changes.

Since the last deployment occurred in September 2026 and the authentication work has not yet been merged into `main`, the version currently published on GitHub Pages is expected to be the first-phase application (tasks, statistics, categories and themes) without the login pages. The

exact Pages URL and source branch are set in the repository settings and were not captured in the evidence.

7.6 Release Workflow

The release workflow followed by the team, and the recommended final step, are:

1. Develop each feature on its own branch and push it to GitHub.
2. Integrate branches and verify the result locally with Docker Compose and the Selenium suite (16 passed).
3. Open a pull request to `main`, review it and merge it. (For the second-phase work this step is still pending – see Section 2.8.)
4. Jenkins checks out `main`, validates the required files and archives the build artifacts.
5. GitHub Pages republishes the static site from the configured branch.
6. Optionally, build and push the Docker image with `docker compose build web` and `docker compose push web` so that others can run the exact same image. The README documents this step; whether the image was actually pushed to Docker Hub is not shown in the evidence.

The repository currently has **no tags or GitHub Releases** (Figure 2.1 shows "0 Tags" and "No releases published"). Creating a tag such as `v1.0` after merging the final pull request would give the evaluated version a permanent, named reference.

7.7 Build Artifacts

The project produces two kinds of artifacts:

- **Jenkins archived artifacts** – each successful build stores `index.html`, `style.css`, `script.js` and `README.md` (main-branch Jenkinsfile) with MD5 fingerprints. They can be downloaded from the build page (the "Artifacts" link in Figure 4.3) and traced across builds.
- **Docker images** – the web image `sohails2409/student-task-manager:latest` and the locally built test-runner image. The web image is a self-contained, deployable artifact of the application.

CHAPTER 8: OPERATIONS AND MONITORING

8.1 Running Containers

During operation, the state of the containers is inspected with `docker ps` (Figure 5.3), `docker compose ps` and the **Containers** view of Docker Desktop. The observed state showed both long-running services up: the `web` container for 12 minutes and the `selenium` container for 3 minutes with status **healthy**. Container names follow the Compose pattern `<project>-<service>-<index>`, for example `devop_project-web-1`.

8.2 Docker Compose Lifecycle

Table 8.1 describes the lifecycle of the Compose application as used in the project.

Table 8.1: Docker Compose lifecycle

Phase	Command	Effect
Build	docker compose build [web]	Builds images from the Dockerfiles; unchanged layers are reused from cache
Start (app only)	docker compose up --build [-d]	Creates the network and starts web; Nginx serves on localhost:8080
Start (tests)	docker compose --profile test run --build --rm tests	Starts web and selenium, waits for health, runs the suite, removes the test container
Inspect	docker ps / docker compose ps / docker compose logs	Shows status, health, ports and logs
Stop	Ctrl+C or docker compose stop	Stops containers without removing them
Tear down	docker compose down	Stops and removes containers and the project network

Note that after a `run` of the test profile, the `selenium` container keeps running (as seen in Figure 5.3). It is removed by `docker compose down`; if it was started only through the test profile, `docker compose --profile test down` ensures that profile services are included.

8.3 Container Health and Application Availability

The only automated health check in the project is the Selenium Grid check (`curl -f http://localhost:4444/status`), which Docker evaluates every five seconds and which gates the start of the test runner. The `web` service has no health check; its availability is verified by opening `http://localhost:8080` in a browser, and implicitly by every Selenium test, each of which loads pages from `http://web`. A failed page load would cause the affected tests to

fail, so the 16 passing tests also demonstrate that the web container was available throughout the run.

8.4 Jenkins Build History and Status Monitoring

Jenkins provides the project's build monitoring. The dashboard (Figure 4.1) shows, for each job, a status icon (S), a weather icon (W) summarising recent stability, the time since the last success and last failure, and the last build duration. For Student-Task-Manager-CI the status is green (successful) and the weather icon is sunny, indicating that recent builds have been stable. The build page (Figure 4.3) shows each stage's result and duration, and the console output (Figure 4.4) provides the full log of every step for troubleshooting.

8.5 Logs and Troubleshooting

The following log sources were used:

- **Jenkins console output** – the full step-by-step log of every build, including each `bat` command and its working directory.
- **docker compose logs** and the foreground output of `docker compose up` – Nginx access and error logs from the `web` container and Selenium Grid logs from the `selenium` container.
- **Pytest output** – verbose per-test results with `-vv`, an all-outcomes summary with `-rA`, and printed output with `-s`. On failure, Pytest shows the failing assertion or timeout and the traceback.
- **noVNC live view** at `http://localhost:7900` – lets a developer watch the automated Chrome browser to see exactly where a test gets stuck.
- **Browser developer tools** – the Console tab shows the `console.error` messages written by `auth.js` and `script.js` when localStorage cannot be read or written; the Application tab shows the stored keys.

8.6 Common Problems Encountered and Solutions

Table 8.2 lists the problems identified from the project documentation, the commit history and the source code, together with the solutions implemented.

Table 8.2: Problems encountered and solutions implemented

Problem	Cause	Solution implemented
Jenkins pipeline failed (last failure: build #3)	Jenkinsfile problems, including a byte-order mark (BOM) at the start of the file	Commits "Fix Jenkins pipeline" and "Fix Jenkins pipeline BOM"; later builds succeeded
Theme briefly showed light before switching to dark (FOUC)	Theme was applied by script after the page was painted	Inline script in <head> applies data-theme before paint (commit c058ddf)
crypto.subtle unavailable in Selenium tests	Web Crypto requires a secure context; http://web is not secure	Hand-written demonstration hash (FNV-1a), documented as not secure
Tests could start before the browser was ready	Selenium Grid needs time to initialise	Compose health check on /status and depends_on: service_healthy
Timing-dependent test failures	Signup redirects after a 1 s delay; pages load asynchronously	WebDriverWait explicit waits instead of sleep
Tests affecting each other through stored data	localStorage persists within a browser session	New remote Chrome session per test in the browser fixture
Two members needed to work in parallel on dependent features	Tests depend on the IDs and messages of unfinished auth pages	Written contract of IDs, messages and keys in 00 shared context.md
Built-in browser validation would replace the app's messages	HTML required/type=email trigger native pop-ups	novalidate on all forms; validation in auth.js
Chrome crashes in containers with small shared memory	Docker's default /dev/shm is 64 MB	shm_size: 2gb on the selenium service
Old data from the first phase	Earlier versions used keys studentTheme, studentUsers, studentCurrentUserId	auth.js migrates legacy keys; script.js keeps studentTheme in sync

8.7 Limitations of the Current Monitoring Setup

The monitoring in this project is basic and appropriate for a laboratory demonstration. It is important to be clear about what has **not** been implemented:

- There is no metrics collection or dashboarding (for example Prometheus and Grafana), no centralised log aggregation (for example the ELK stack), and no alerting.
- The `web` service has no Docker health check, and there is no external uptime monitoring of GitHub Pages or the container.
- Application errors are logged only to each user's browser console and are never reported to the team.

- Jenkins notifications (e-mail or chat) are not configured; build status must be checked on the dashboard.
- Selenium results are seen in the terminal of whoever runs them; they are not stored or trended over time.

These gaps are addressed in the future scope (Chapter 9).

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Project Achievements

The project achieved its objectives. The team built a responsive, framework-free Student Task Manager with task creation, completion, deletion, categories, live statistics, persistent dark and light themes, and per-user task lists, and extended it with client-side signup, login, logout, protected pages, profile editing and password change. Around this application the team implemented a DevOps workflow using Git and GitHub with feature branches and three merged pull requests; a Jenkins declarative pipeline that checks out, validates and archives the project on a Windows agent; a Docker image based on Nginx; a Docker Compose environment with application, browser and test-runner services, profiles and health checks; and an automated Selenium/Pytest suite of 16 end-to-end tests, **all of which passed in 34.01 seconds** in the observed run.

9.2 DevOps Concepts Demonstrated

Table 9.1: DevOps concepts and how they were demonstrated

Concept	Demonstration in this project
Version control	Complete history in Git; 23 commits across six branches
Collaboration and code review	Feature branches and pull requests #1–#3 on GitHub
Continuous integration	Jenkins job Student-Task-Manager-CI with Checkout, Validate and Archive stages
Pipeline as code	Jenkinsfile committed to the repository
Artifact management	Archived, fingerprinted build artifacts; tagged Docker image
Containerisation	nginx:alpine image serving the application
Infrastructure as code	Dockerfile, tests/Dockerfile and docker-compose.yml under version control
Automated testing	16 Selenium end-to-end tests run with Pytest in containers
Deployment	Local Docker deployment at localhost:8080; GitHub Pages static hosting
Contract-first parallel development	Shared contract enabled tests to be written before the implementation

9.3 Technical Learning Outcomes

- Using Git branches, merges and pull requests to coordinate a team, and the value of clear commit messages.
- Writing a declarative Jenkins pipeline, using Windows `bat` steps and exit codes to control build results, and archiving artifacts with fingerprints.
- Writing Dockerfiles, understanding image layers and build contexts, publishing ports, and using Compose networks, service-name DNS, profiles and health checks.
- Designing end-to-end tests from requirements, using Selenium locators and explicit waits, and isolating tests with Pytest fixtures.
- Structuring a client-side application, using localStorage safely, preventing HTML injection, and understanding the limits of browser-side security.

9.4 Limitations

The following limitations apply to the implemented system:

- **Security:** authentication is client-side and uses an unsalted 32-bit FNV-1a hash. Anyone with access to the browser can read or modify accounts, sessions and tasks. It must not be used for real credentials.
- **Data storage:** data lives only in one browser's localStorage. It is not synchronised across devices, is lost if site data is cleared, and is limited to a few megabytes.
- **CI coverage:** the executed Jenkins pipeline only checks that three files exist and archives them. The Docker Build and Selenium Tests stages exist only on the integration branch and have not been run by Jenkins.
- **Integration:** the second-phase work has not yet been merged into `main` through a pull request, so GitHub Pages and the Jenkins job still reflect the first-phase application.
- **Testing scope:** one browser, one machine, a single observed run, and functional tests only.
- **Deployment and monitoring:** deployment is local only, and monitoring is limited to Docker status, one health check and the Jenkins dashboard.

9.5 Future Enhancements

The following improvements are **proposed** and have **not** been implemented:

- **Backend authentication:** move registration and login to a server (for example Node.js/Express or Python/Flask) with session cookies or tokens.
- **Database integration:** store users and tasks in a database server such as PostgreSQL or MongoDB, enabling access from any device.
- **Stronger password security:** hash passwords on the server with a slow, salted algorithm such as bcrypt or Argon2, serve the site over HTTPS, and add rate limiting.
- **Cloud deployment:** deploy the container image to a cloud platform (for example AWS, Azure or Google Cloud) or a Kubernetes cluster.
- **More comprehensive automated tests:** add JavaScript unit tests, task completion and deletion tests, accessibility tests, and cross-browser runs using Selenium Grid with Firefox and Edge nodes.
- **Improved monitoring:** add a health check to the web service, uptime monitoring, centralised logs, metrics with Prometheus and Grafana, and Jenkins notifications.
- **More advanced CI/CD:** merge and activate the extended Jenkinsfile so that every push builds the image and runs the Selenium suite; publish JUnit test reports in Jenkins; trigger builds with GitHub webhooks; build pull requests with a multibranch pipeline; and push versioned images to a registry automatically.

9.6 Conclusion

Student Task Manager demonstrates that even a small application benefits greatly from a disciplined DevOps workflow. Version control with feature branches allowed three members to work independently; a written contract allowed implementation and tests to be developed in parallel; Docker made the runtime and test environment reproducible with a single command; and automated browser tests provided objective evidence that the integrated application behaves as specified. The project has also clearly identified the next steps – merging the integration branch, running the extended pipeline in Jenkins, and replacing the demonstration-level client-side authentication with a secure backend – that would move it from a laboratory demonstration towards a production-ready system.

REFERENCES

[1] Git Project, "Git Documentation." Available: https://git-scm.com/doc

[2] S. Chacon and B. Straub, Pro Git, 2nd ed., Apress. Available: https://git-scm.com/book/en/v2

[3] GitHub, "About pull requests," GitHub Docs. Available: https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

[4] GitHub, "What is GitHub Pages?," GitHub Docs. Available: https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages

[5] Jenkins Project, "Pipeline," Jenkins User Documentation. Available: https://www.jenkins.io/doc/book/pipeline/

[6] Jenkins Project, "Pipeline Syntax (Declarative Pipeline)." Available: https://www.jenkins.io/doc/book/pipeline/syntax/

[7] Jenkins Project, "Installing Jenkins on Windows." Available: https://www.jenkins.io/doc/book/installing/windows/

[8] Docker Inc., "Dockerfile reference," Docker Docs. Available: https://docs.docker.com/reference/dockerfile/

[9] Docker Inc., "Docker Compose," Docker Docs. Available: https://docs.docker.com/compose/

[10] Docker Inc., "Using profiles with Compose," Docker Docs. Available: https://docs.docker.com/compose/how-tos/profiles/

[11] Docker Inc., "Networking in Compose," Docker Docs. Available: https://docs.docker.com/compose/how-tos/networking/

[12] Docker Hub, "nginx – Official Image." Available: https://hub.docker.com/_/nginx

[13] Selenium Project, "WebDriver," Selenium Documentation. Available: https://www.selenium.dev/documentation/webdriver/

[14] Selenium Project, "Waiting Strategies," Selenium Documentation. Available: https://www.selenium.dev/documentation/webdriver/waits/

[15] SeleniumHQ, "docker-selenium" (standalone-chrome images), GitHub. Available: https://github.com/SeleniumHQ/docker-selenium

[16] pytest Developers, "pytest Documentation." Available: https://docs.pytest.org/en/stable/

[17] pytest Developers, "About fixtures." Available: https://docs.pytest.org/en/stable/explanation/fixtures.html

[18] MDN Web Docs, "Window: localStorage property." Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[19] MDN Web Docs, "Using CSS custom properties (variables)." Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties

[20] MDN Web Docs, "Secure contexts." Available: https://developer.mozilla.org/en-US/docs/Web/Security/Secure_Contexts

[21] MDN Web Docs, "SubtleCrypto." Available: https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto

[22] Nginx, "nginx documentation." Available: https://nginx.org/en/docs/